Merko — Enterprise Architecture & Product Documentation

> **Classification:** Internal Technical Reference
> **Version:** 1.0.0
> **Status:** Draft for Review
> **Stack:** Next.js 15 · Node.js · PostgreSQL · Prisma · Cloudinary · Razorpay

---

Table of Contents

---

1. Product Requirements Document

1.1 Executive Summary

Merko is a Flipkart-style customizable product marketplace that enables customers to browse, personalize, and purchase printed and customized physical goods — ID cards, mugs, T-shirts, banners, and more. The platform differentiates itself through a dynamic, admin-configurable

customization engine that allows product managers to define custom fields, input types, and preview logic without any code changes.

1.2 Problem Statement

Problem	Impact
Local printing businesses have no digital storefront	Lost revenue, no reach beyond walk-ins
Customers cannot preview customized products before ordering	High return rates, low conversion
Admin teams must involve developers to add new product fields	Slow time-to-market, high operational cost
No centralized order + production tracking for print businesses	Fulfillment errors, poor customer experience

1.3 Product Vision

> *"Make professional customized printing accessible to everyone — browse, personalize, preview, and order in under 5 minutes."*

1.4 Success Metrics

Metric	Target (6 months)
Registered users	10,000+
Monthly active orders	2,000+
Average order value	₹350+
Cart-to-checkout conversion	⩾ 35%
Page load time (LCP)	< 2.5s
Uptime	⩾ 99.9%
Admin field setup time	< 10 minutes (no code)

1.5 Scope

In Scope (v1 MVP)

- Product catalog with categories and search
- Dynamic customization engine (admin-configured fields)

- Live design preview for customizable products
- File upload (photos, logos, designs)
- Cart and multi-product checkout
- Razorpay payment integration (UPI, Card, NetBanking)
- Order tracking with status updates
- Customer notifications (Email + WhatsApp + Push)
- Admin panel: product, order, user, coupon management
- JWT authentication with refresh tokens

Out of Scope (v2+)

- Vendor multi-seller portal
- AI design suggestion engine
- Mobile native app (iOS/Android)
- Loyalty/referral programs
- Bulk B2B order portal

---

2. User Roles

2.1 Role Matrix

Capability	Guest	Customer	Admin	Super Admin
Browse products	✓	✓	✓	✓
Search & filter	✓	✓	✓	✓
Customize product	✓ (preview only)	✓	✓	✓
Add to cart	✗	✓	✗	✗
Checkout & pay	✗	✓	✗	✗
Track orders	✗	✓	✓	✓
Manage own profile	✗	✓	✓	✓
Write reviews	✗	✓	✗	✗
Manage products	✗	✗	✓	✓
Manage custom fields	✗	✗	✓	✓
Manage orders	✗	✗	✓	✓

Capability	Guest	Customer	Admin	Super Admin
Manage users	✗	✗	✗	✓
Manage coupons	✗	✗	✓	✓
View analytics	✗	✗	✓	✓
Manage admins	✗	✗	✗	✓
System configuration	✗	✗	✗	✓

2.2 Role Definitions

Guest

- Unauthenticated visitor
- Can browse, search, preview customizations
- Cannot persist cart or complete purchase
- Prompted to register at cart/checkout

Customer

- Registered, verified user
- Full shopping journey access
- Can manage saved addresses, order history, saved designs
- Can submit reviews post-delivery

Admin

- Operations team member
- Manages product catalog, customization fields, orders, gallery, coupons
- Cannot manage other admin accounts
- Access scoped to operational data

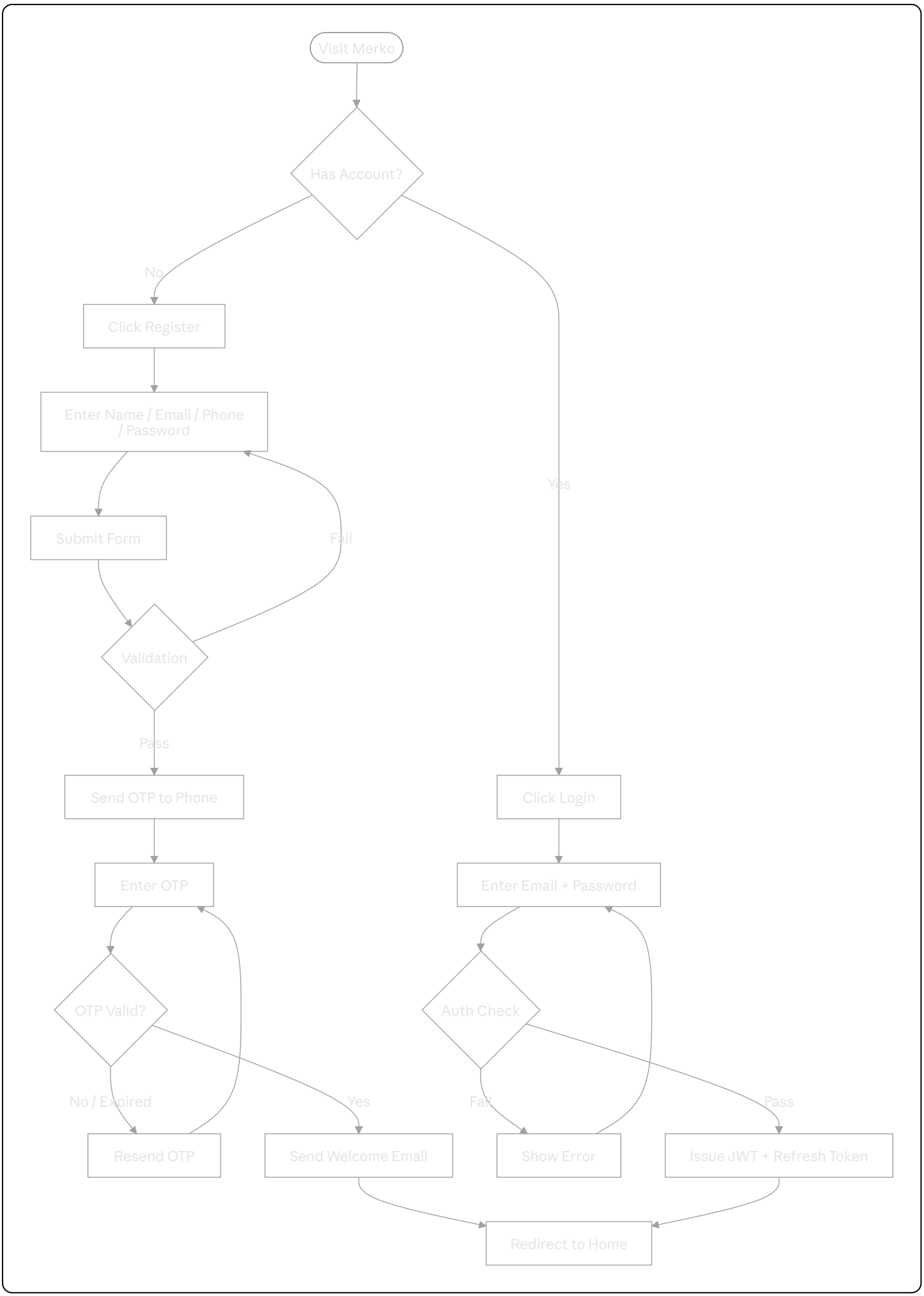
Super Admin

- Full system access
- Manages admin accounts and roles
- Access to system configuration, raw analytics, audit logs
- Can impersonate users for support purposes (logged)

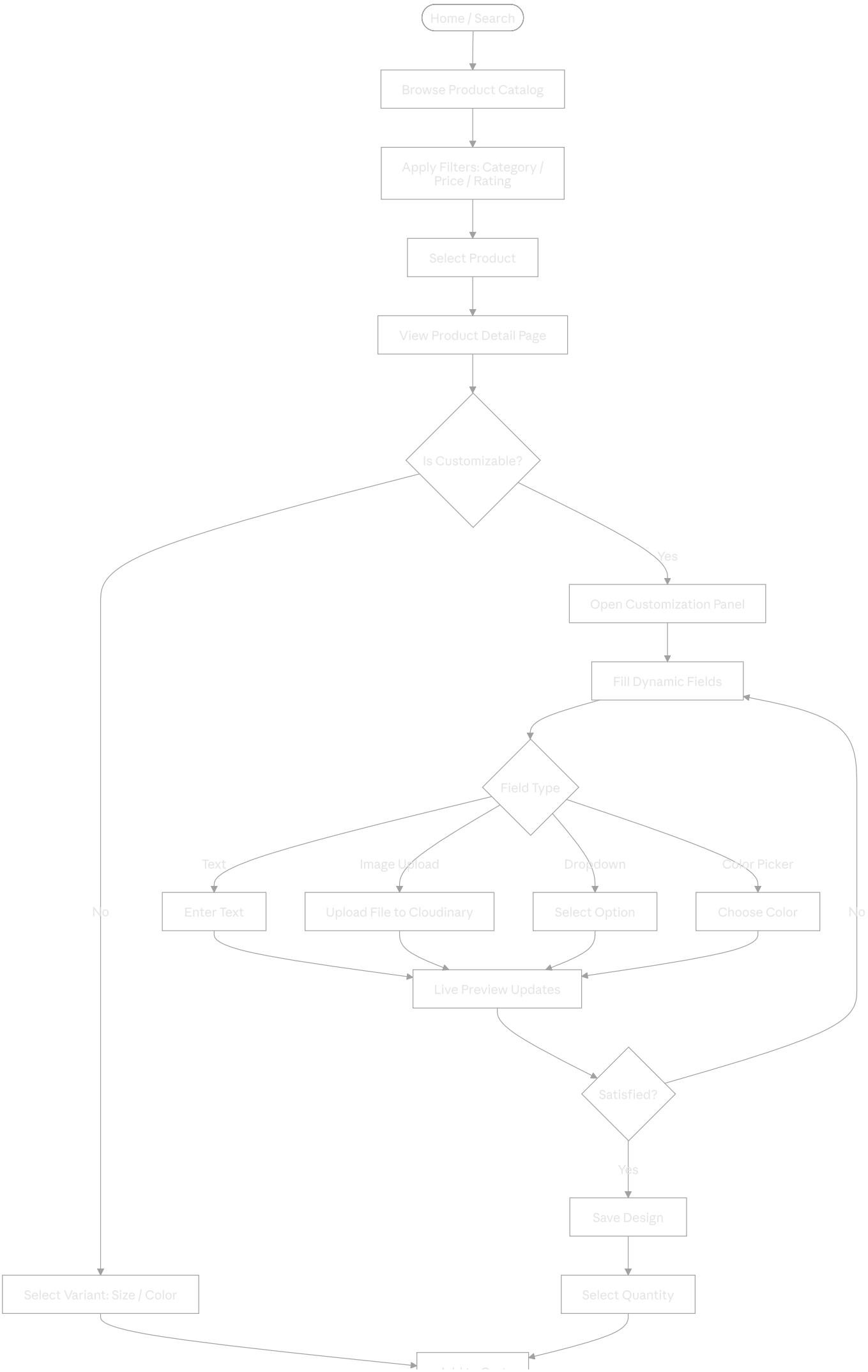
---

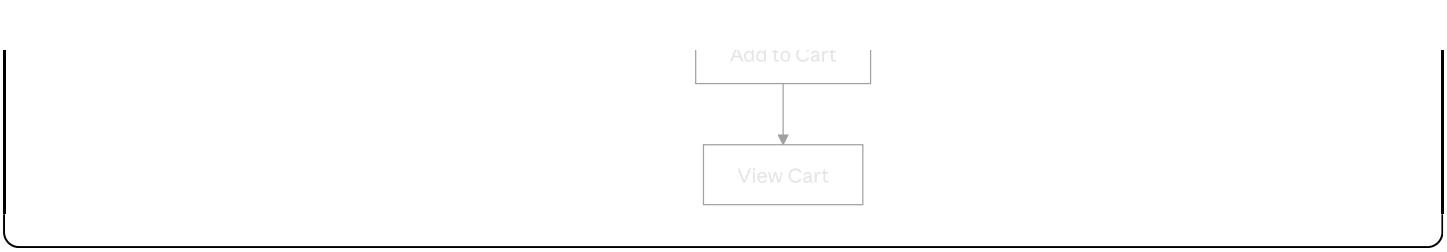
3. User Journeys

3.1 Customer Registration & Onboarding

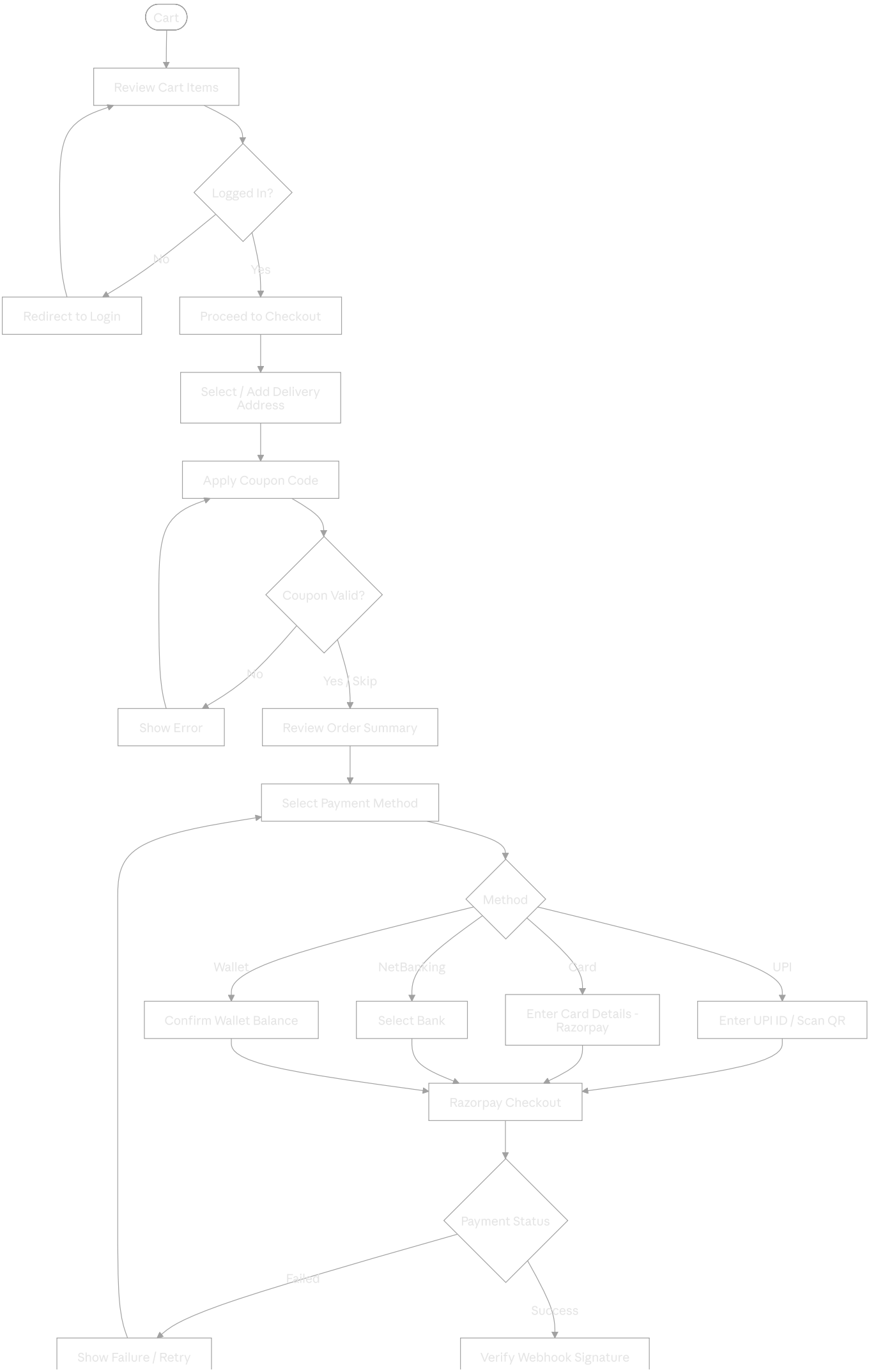

Visit Merko
Has Account?
No
Yes
Click Register
Enter Name / Email / Phone / Password
Submit Form
Fail
Validation
Pass
Send OTP to Phone
Enter OTP
OTP Valid?
No / Expired
Yes
Resend OTP
Send Welcome Email
Click Login
Enter Email + Password
Auth Check
Fail
Pass
Show Error
Issue JWT + Refresh Token
Redirect to Home

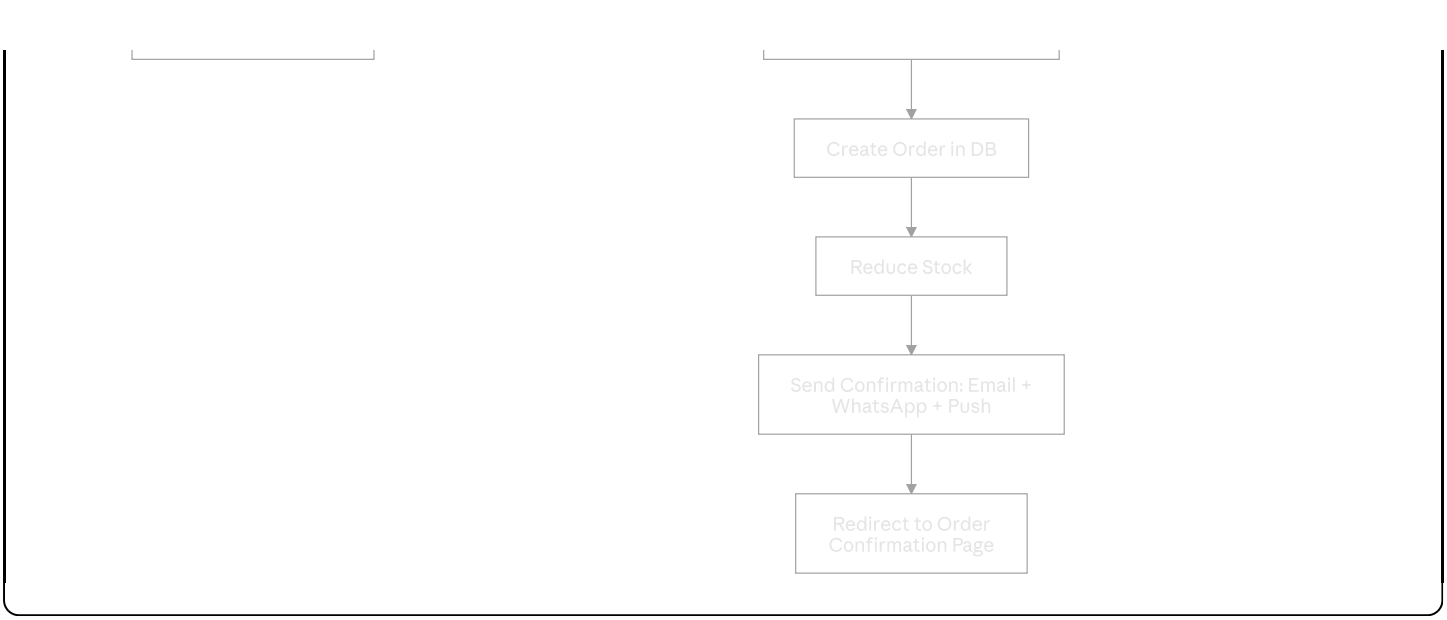
3.2 Product Discovery & Customization


Home / Search
Browse Product Catalog
Apply Filters: Category / Price / Rating
Select Product
View Product Detail Page
Is Customizable?
Yes
No
Open Customization Panel
Fill Dynamic Fields
Field Type
Text
Image Upload
Dropdown
Color Picker
Enter Text
Upload File to Cloudinary
Select Option
Choose Color
Live Preview Updates
Satisfied?
No
Yes
Save Design
Select Variant: Size / Color
Select Quantity

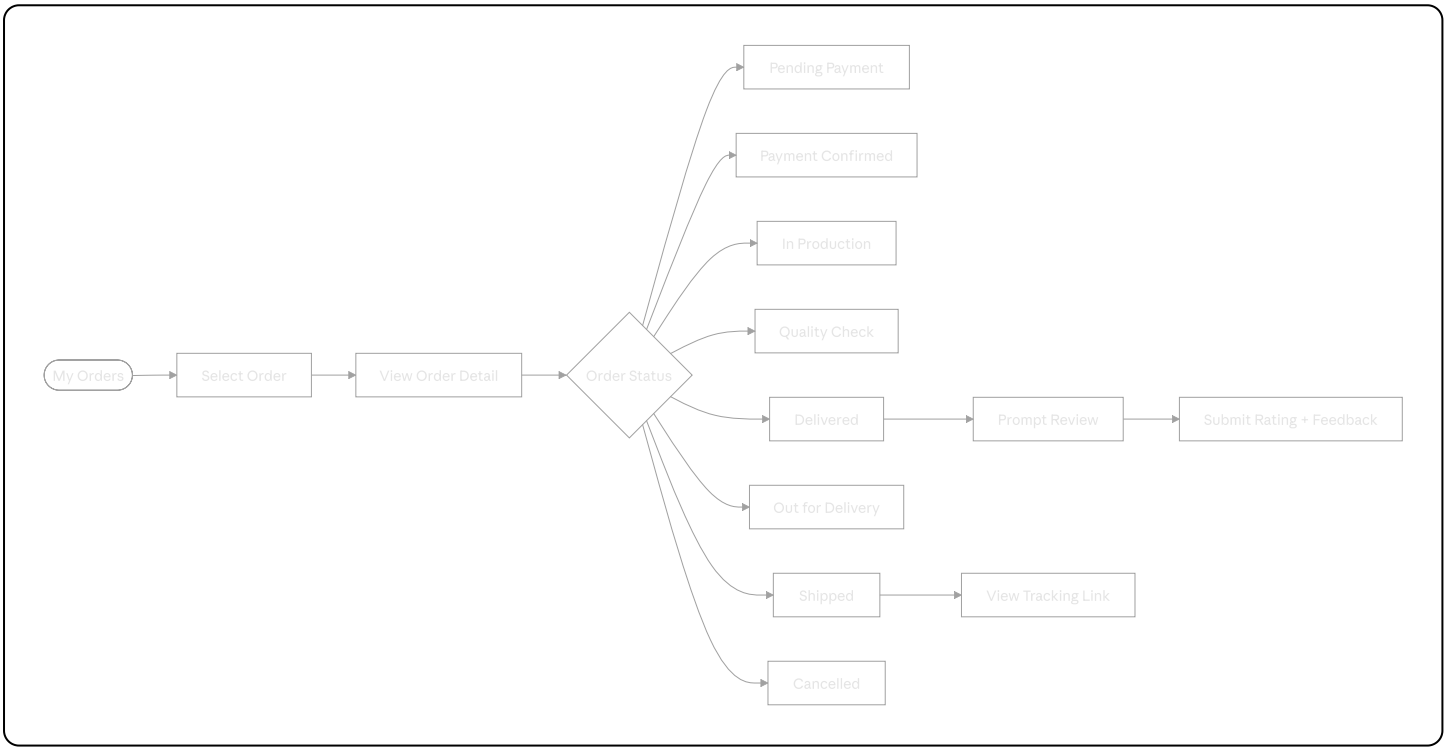

Add to Cart
View Cart

3.3 Checkout & Payment


Cart
Review Cart Items
Logged In?
No
Yes
Redirect to Login
Proceed to Checkout
Select / Add Delivery Address
Apply Coupon Code
Coupon Valid?
No
Yes / Skip
Show Error
Review Order Summary
Select Payment Method
Method
Wallet
NetBanking
Card
UPI
Confirm Wallet Balance
Select Bank
Enter Card Details - Razorpay
Enter UPI ID / Scan QR
Razorpay Checkout
Payment Status
Failed
Success
Show Failure / Retry
Verify Webhook Signature

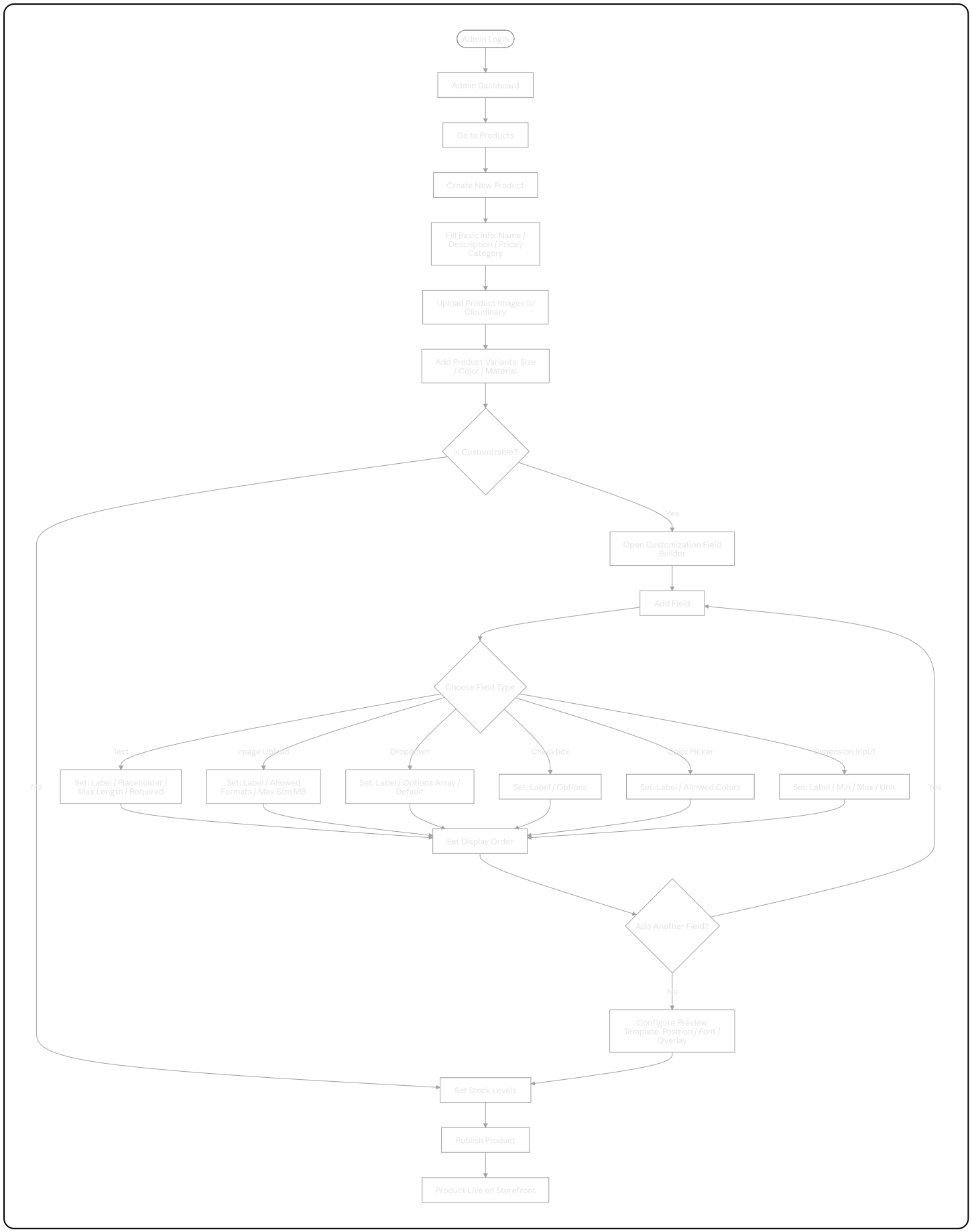


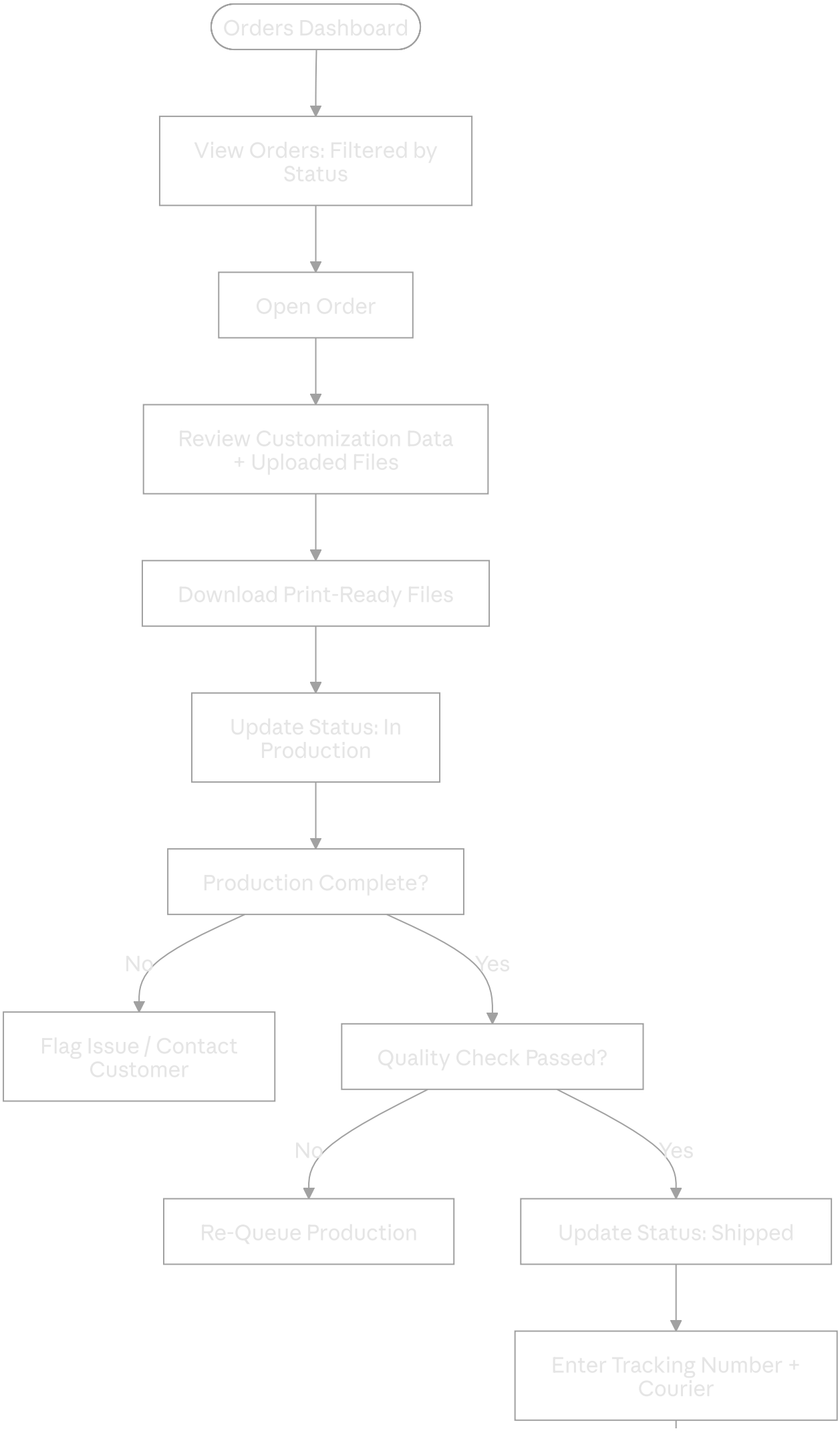
3.4 Order Tracking

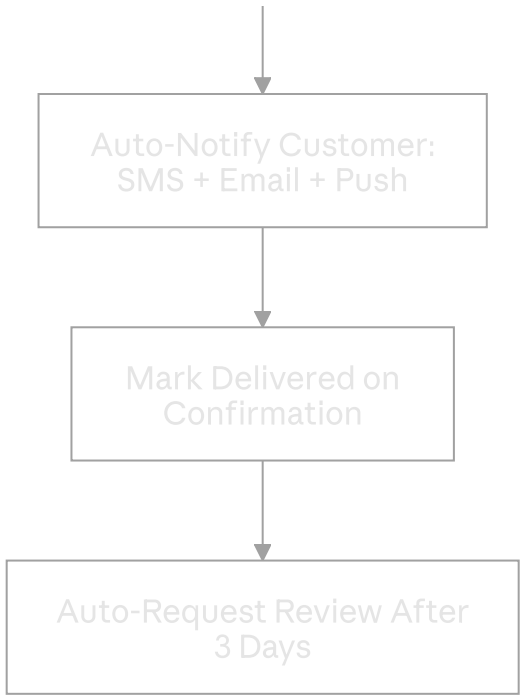


4. Admin Journeys

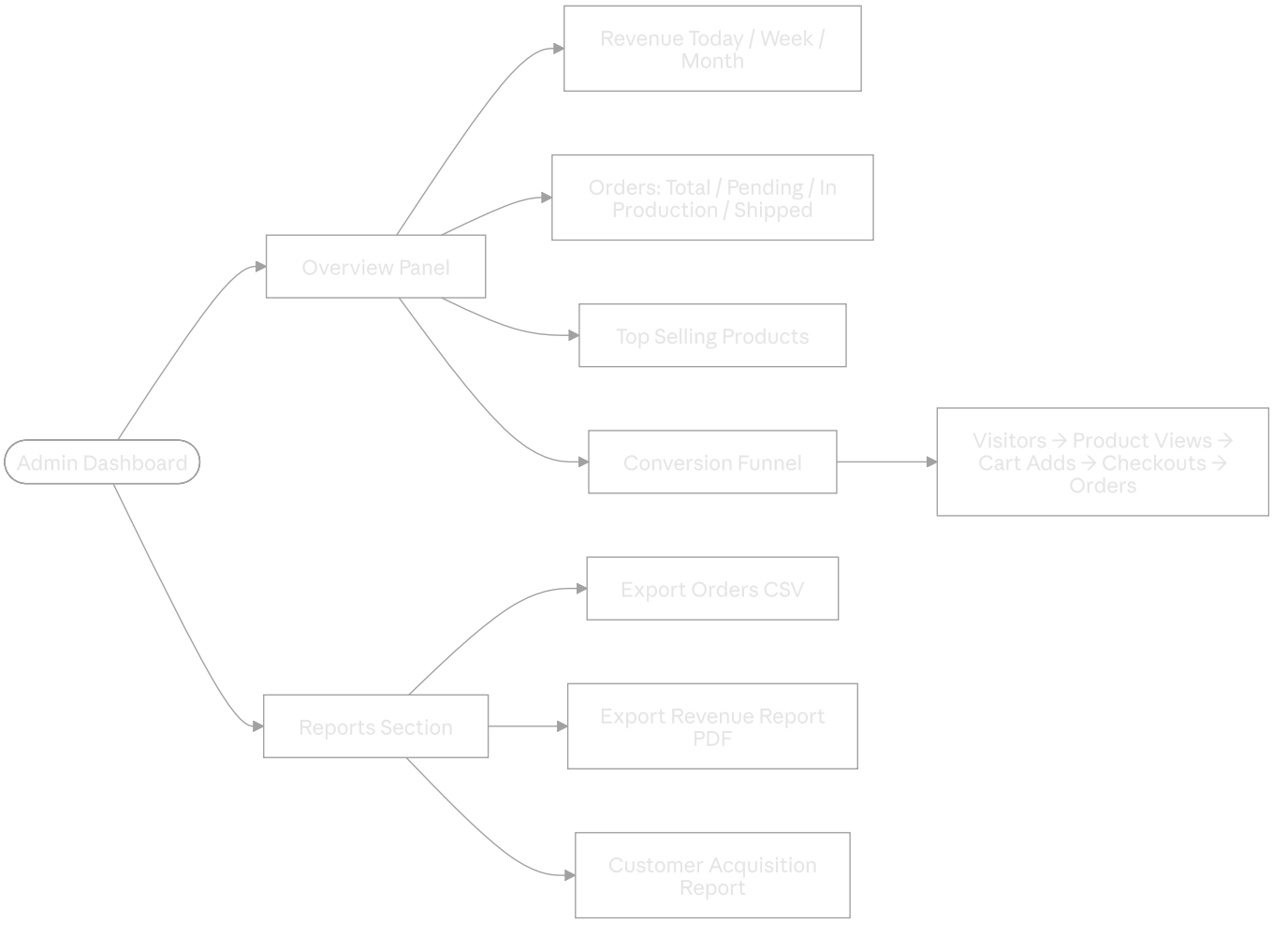
4.1 Product & Customization Field Management




Orders Dashboard
View Orders: Filtered by Status
Open Order
Review Customization Data + Uploaded Files
Download Print-Ready Files
Update Status: In Production
Production Complete?
No
Yes
Flag Issue / Contact Customer
Quality Check Passed?
No
Yes
Re-Queue Production
Update Status: Shipped
Enter Tracking Number + Courier



4.3 Analytics & Reporting



5. Functional Requirements

5.1 Authentication Module

ID	Requirement	Priority
AUTH-01	User registration with name, email, phone, password	P0
AUTH-02	Phone OTP verification on registration	P0
AUTH-03	Login with email + password	P0
AUTH-04	JWT access token (15-min) + refresh token (7-day)	P0
AUTH-05	Refresh token rotation — invalidate old on use	P0
AUTH-06	Forgot password via email OTP	P1
AUTH-07	Google OAuth SSO	P2
AUTH-08	Admin login with 2FA (TOTP)	P1
AUTH-09	Session revocation from admin panel	P1

5.2 Product & Catalog Module

ID	Requirement	Priority
PROD-01	Product listing with pagination (20/page)	P0
PROD-02	Category hierarchy (parent/child)	P0
PROD-03	Full-text product search (name + description)	P0
PROD-04	Filter by: category, price range, rating, availability	P0
PROD-05	Sort by: price asc/desc, newest, best-selling	P0
PROD-06	Product detail page with image gallery	P0
PROD-07	Product variants (size, color, material) with individual pricing/stock	P0
PROD-08	Related products section	P1
PROD-09	Recently viewed products (local + DB)	P1
PROD-10	Product availability / stock status real-time	P0

5.3 Dynamic Customization Engine

ID	Requirement	Priority
CUST-01	Admin defines custom fields per product without code changes	P0
CUST-02	Field types: text, textarea, number, image upload, dropdown, checkbox, radio, color picker, dimension	P0
CUST-03	Field-level validation: required, min/max length, format, file type	P0
CUST-04	Drag-and-drop field ordering by admin	P1
CUST-05	Live preview canvas updates on every field change	P0
CUST-06	Preview template: define text position, font, image area per product type	P0
CUST-07	Save design to user account for reuse	P1
CUST-08	Customization data serialized as JSONB and stored with order item	P0
CUST-09	Admin can preview customer's customization data from order panel	P0
CUST-10	Print-ready file generation (PDF/PNG export) with customer data injected	P1

5.4 Cart & Order Module

ID	Requirement	Priority
CART-01	Persistent cart (DB-backed for auth users)	P0
CART-02	Guest cart (localStorage, migrate on login)	P1
CART-03	Add / update quantity / remove cart items	P0
CART-04	Cart item includes product variant + customization snapshot	P0
CART-05	Dynamic pricing: base price + variant delta + customization surcharge	P0

ID	Requirement	Priority
CART-06	Coupon code application with live discount calculation	P0
ORD-01	Order creation atomically (cart → order + order_items)	P0
ORD-02	Order status lifecycle: pending → confirmed → production → shipped → delivered	P0
ORD-03	Order cancellation (before production) with auto-refund trigger	P1
ORD-04	Order history with filters and pagination	P0
ORD-05	Re-order functionality (copy last order to cart)	P2

5.5 Payment Module

ID	Requirement	Priority
PAY-01	Razorpay checkout: UPI, card, netbanking, wallets	P0
PAY-02	Webhook signature verification for all payment events	P0
PAY-03	Idempotency key on all payment creation requests	P0
PAY-04	Refund initiation from admin panel	P1
PAY-05	Payment failure retry (3 attempts before abandonment)	P0
PAY-06	Invoice PDF auto-generation and email attachment	P1

5.6 Admin Module

ID	Requirement	Priority
ADM-01	Admin dashboard: revenue, orders, top products KPIs	P0
ADM-02	Product CRUD with image upload and variant management	P0
ADM-03	Dynamic field builder per product (no-code)	P0
ADM-04	Order management with status pipeline	P0
ADM-05	Customer list with order history and account status	P1
ADM-06	Coupon creation: percentage, flat, free-shipping types	P1
ADM-07	Gallery management (upload portfolio/completed work)	P1
ADM-08	Review moderation (approve/reject/reply)	P1

ID	Requirement	Priority
ADM-09	Export orders and revenue as CSV/PDF	P1
ADM-10	Notification broadcast to all customers	P2

6. Non-Functional Requirements

6.1 Performance

Metric	Target	Measurement
Largest Contentful Paint (LCP)	< 2.5s	Vercel Speed Insights
First Input Delay (FID)	< 100ms	Core Web Vitals
API P95 response time	< 300ms	Railway Metrics
API P99 response time	< 800ms	Railway Metrics
Database query P95	< 50ms	pg_stat_statements
File upload processing	< 3s (⩽5MB)	Cloudinary API
Customization preview render	< 200ms	Client-side

6.2 Scalability

Dimension	Target
Concurrent users	10,000+
Peak orders/hour	500
File uploads/hour	2,000
Database connections (pooled)	100 max (PgBouncer)
Storage growth tolerance	100GB/year Cloudinary

6.3 Security

Requirement	Standard
Password hashing	bcrypt, cost factor 12

Requirement	Standard
Transport encryption	TLS 1.3, HSTS
JWT signing	HS256, secrets rotated quarterly
File upload validation	MIME type + magic bytes + virus scan
SQL injection prevention	Prisma parameterized queries only
XSS prevention	CSP headers, sanitized input
Payment data	PCI-DSS scope minimized — no card data stored
Rate limiting	100 req/min general; 5/min auth endpoints

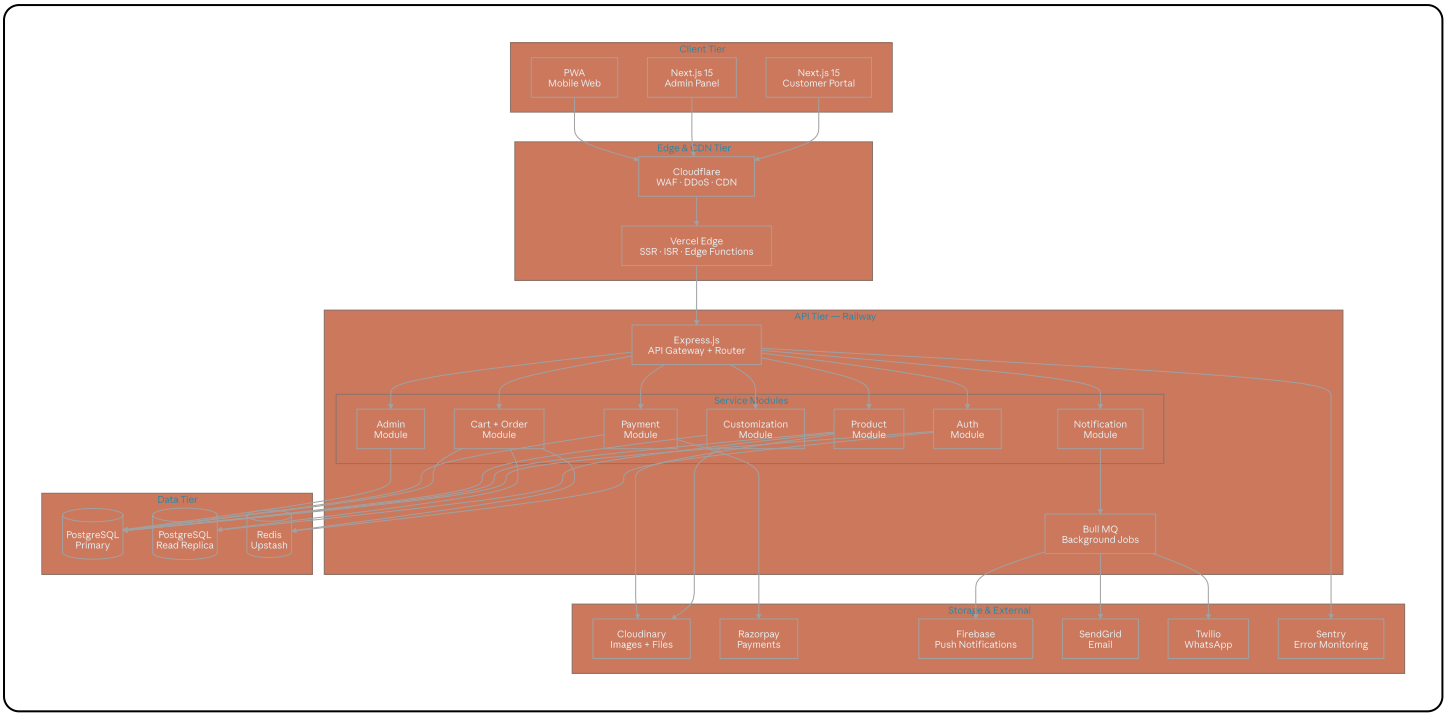
6.4 Availability & Reliability

Metric	Target
Uptime SLA	99.9% (< 8.7 hrs downtime/year)
RTO (Recovery Time Objective)	< 1 hour
RPO (Recovery Point Objective)	< 15 minutes
Database backup	Daily full + continuous WAL
Deployment rollback	< 5 minutes via Vercel/Railway

6.5 Compliance & Accessibility

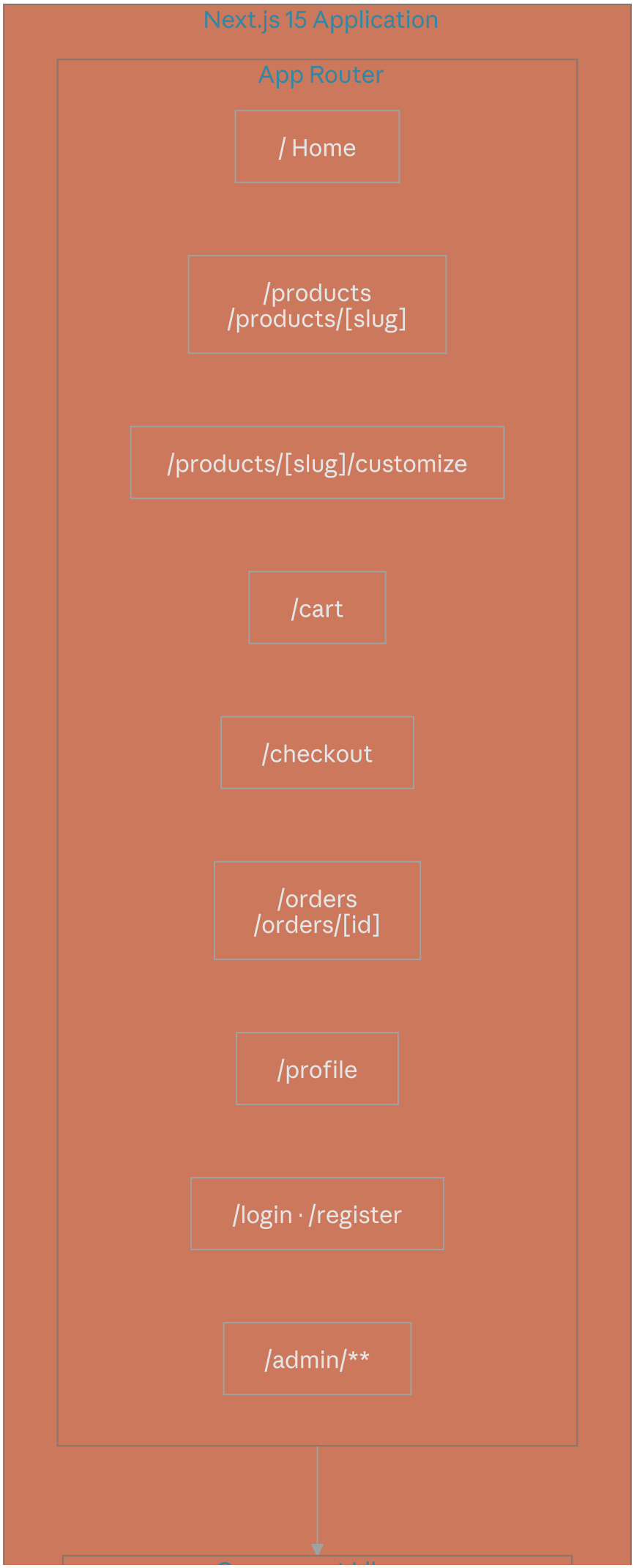
- WCAG 2.1 AA accessibility compliance
- GDPR-compatible data handling (data export / deletion on request)
- Indian IT Act 2000 compliance
- Cookie consent banner
- Privacy policy and terms of service

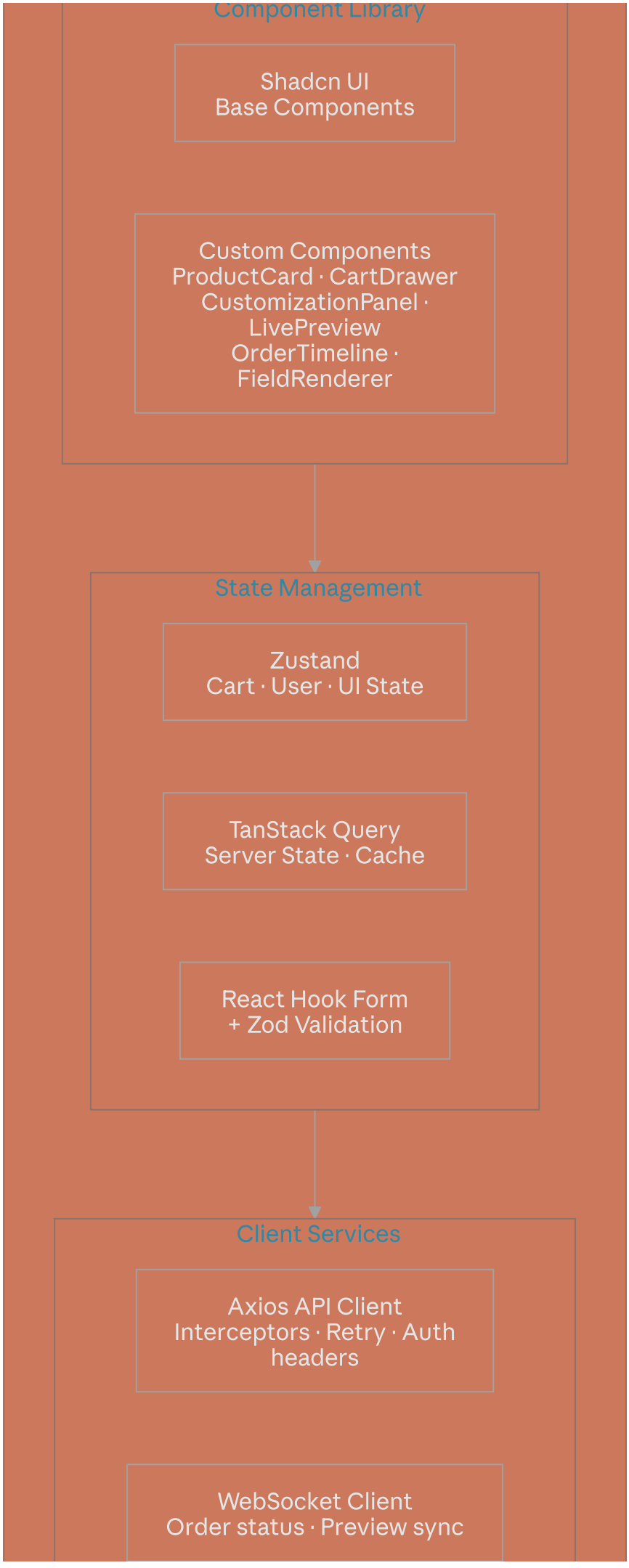
7. High-Level Architecture



8. Detailed Component Architecture

8.1 Frontend Architecture


Next.js 15 Application
App Router
/ Home
/products
/products/[slug]
/products/[slug]/customize
/cart
/checkout
/orders
/orders/[id]
/profile
/login · /register
/admin/**


Component Library
Shadcn UI
Base Components
Custom Components
ProductCard · CartDrawer
CustomizationPanel ·
LivePreview
OrderTimeline ·
FieldRenderer
State Management
Zustand
Cart · User · UI State
TanStack Query
Server State · Cache
React Hook Form
+ Zod Validation
Client Services
Axios API Client
Interceptors · Retry · Auth
headers
WebSocket Client
Order status · Preview sync

9. Database ER Diagram

```
erDiagram
    users {
        uuid id PK
        string email UK
        string phone UK
        string password_hash
        enum role
        boolean is_verified
        boolean is_active
        timestamptz created_at
        timestamptz updated_at
    }

    addresses {
        uuid id PK
        uuid user_id FK
        string full_name
        string phone
        string line1
        string line2
        string city
        string state
        string pincode
        boolean is_default
    }

    categories {
        uuid id PK
        uuid parent_id FK
        string name
        string slug UK
        string image_url
        int sort_order
        boolean is_active
    }

    products {
        uuid id PK
        uuid category_id FK
        string name
        string slug UK
        text description
        decimal base_price
        string sku UK
        boolean is_customizable
        jsonb customization_schema
        jsonb images
        boolean is_active
        timestamptz created_at
    }
```

```
product_variants {
    uuid id PK
    uuid product_id FK
    string name
    jsonb options
    decimal price_delta
    int stock
    string sku UK
    boolean is_active
}

design_templates {
    uuid id PK
    uuid product_id FK
    string name
    jsonb canvas_config
    int width_px
    int height_px
}

user_designs {
    uuid id PK
    uuid user_id FK
    uuid product_id FK
    string name
    jsonb customization_data
    string preview_url
    timestamptz created_at
}

carts {
    uuid id PK
    uuid user_id FK UK
    timestamptz updated_at
}

cart_items {
    uuid id PK
    uuid cart_id FK
    uuid variant_id FK
    int quantity
    jsonb customization_data
    decimal unit_price
}

coupons {
    uuid id PK
    string code UK
    enum type
    decimal value
    decimal min_order_value
    int usage_limit
```

```
    int used_count
    timestamptz valid_from
    timestamptz valid_to
    boolean is_active
}

orders {
    uuid id PK
    string order_number UK
    uuid user_id FK
    uuid address_id FK
    uuid coupon_id FK
    enum status
    decimal subtotal
    decimal discount_amount
    decimal shipping_charge
    decimal total_amount
    text customer_notes
    timestamptz created_at
}

order_items {
    uuid id PK
    uuid order_id FK
    uuid variant_id FK
    int quantity
    decimal unit_price
    jsonb customization_data
    string preview_url
}

payments {
    uuid id PK
    uuid order_id FK UK
    string razorpay_order_id UK
    string razorpay_payment_id
    string razorpay_signature
    decimal amount
    enum status
    string method
    string currency
    timestamptz captured_at
}

reviews {
    uuid id PK
    uuid user_id FK
    uuid product_id FK
    uuid order_id FK
    int rating
    text body
    boolean is_verified_purchase
    boolean is_approved
```

```
        timestamptz created_at
    }

    notifications {
        uuid id PK
        uuid user_id FK
        string title
        text body
        string type
        string entity_type
        uuid entity_id
        boolean is_read
        timestamptz created_at
    }

    activity_logs {
        uuid id PK
        uuid user_id FK
        string action
        string entity_type
        uuid entity_id
        jsonb metadata
        string ip_address
        timestamptz created_at
    }

    users ||--o{ addresses : "has"
    users ||--o{ user_designs : "saves"
    users ||--|| carts : "has"
    users ||--o{ orders : "places"
    users ||--o{ reviews : "writes"
    users ||--o{ notifications : "receives"
    categories ||--o{ categories : "parent of"
    categories ||--o{ products : "contains"
    products ||--o{ product_variants : "has"
    products ||--o{ design_templates : "uses"
    products ||--o{ reviews : "receives"
    carts ||--o{ cart_items : "contains"
    cart_items }o--|| product_variants : "references"
    orders ||--o{ order_items : "contains"
    orders ||--|| payments : "has"
    order_items }o--|| product_variants : "references"
    coupons ||--o{ orders : "applied to"
    addresses ||--o{ orders : "delivered to"
```

---

10. PostgreSQL Schema Design

10.1 Prisma Schema (Abbreviated)

```
// ──────────────────────────────────────
```

```
// ENUMS
// ──────────────────────────────────────────────

enum UserRole        { CUSTOMER  ADMIN  SUPER_ADMIN }
enum OrderStatus     { PENDING  PAYMENT_CONFIRMED  IN_PRODUCTION
                       QUALITY_CHECK  SHIPPED  DELIVERED  CANCELLED  REFUNDED }
enum PaymentStatus   { CREATED  CAPTURED  FAILED  REFUNDED }
enum CouponType      { PERCENTAGE  FLAT_AMOUNT  FREE_SHIPPING }

// ──────────────────────────────────────────────
// USERS & ACCOUNTS
// ──────────────────────────────────────────────

model User {
  id              String    @id @default(uuid())
  email           String    @unique
  phone           String?   @unique
  passwordHash    String
  name            String
  avatarUrl       String?
  role            UserRole  @default(CUSTOMER)
  isVerified      Boolean   @default(false)
  isActive        Boolean   @default(true)
  fcmToken        String?
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  addresses       Address[]
  cart            Cart?
  orders          Order[]
  designs         UserDesign[]
  reviews         Review[]
  notifications   Notification[]
  @@index([email])
  @@index([phone])
}

model Address {
  id          String   @id @default(uuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id])
  fullName    String
  phone       String
  line1       String
  line2       String?
  city        String
  state       String
  pincode     String
```

```
  isDefault   Boolean  @default(false)
  orders      Order[]
}

// ─────────────────────────────────────────────
// CATALOG
// ─────────────────────────────────────────────

model Category {
  id          String     @id @default(uuid())
  parentId    String?
  parent      Category?  @relation("SubCategories", fields: [parentId],
references: [id])
  children    Category[] @relation("SubCategories")
  name        String
  slug        String     @unique
  imageUrl    String?
  sortOrder   Int        @default(0)
  isActive    Boolean    @default(true)
  products    Product[]
}

model Product {
  id                  String            @id @default(uuid())
  categoryId          String
  category            Category          @relation(fields: [categoryId],
references: [id])
  name                String
  slug                String            @unique
  description         String
  basePriceInPaise    Int
  sku                 String            @unique
  isCustomizable      Boolean           @default(false)
  customizationSchema Json?             // FieldDefinition[]
  images              Json              // string[]
  tags                String[]
  isActive            Boolean           @default(true)
  createdAt           DateTime          @default(now())
  updatedAt           DateTime          @updatedAt
  variants            ProductVariant[]
  designTemplates     DesignTemplate[]
  reviews             Review[]
  cartItems           CartItem[]
  @@index([categoryId])
  @@index([slug])
  @@index([isActive])
}
```

```
model ProductVariant {
  id              String      @id @default(uuid())
  productId       String
  product         Product     @relation(fields: [productId], references: [id])
  name            String
  options         Json        // { size: "A4", color: "white" }
  priceDeltaPaise Int         @default(0)
  stock           Int         @default(0)
  sku             String      @unique
  isActive        Boolean     @default(true)
  cartItems       CartItem[]
  orderItems      OrderItem[]
}

// ─────────────────────────────────────────────
// CUSTOMIZATION
// ─────────────────────────────────────────────

// customizationSchema shape (stored as JSONB):
// FieldDefinition {
//   id: string (uuid)
//   type: 'text' | 'textarea' | 'number' | 'image' | 'dropdown'
//         | 'checkbox' | 'radio' | 'color' | 'dimension'
//   label: string
//   placeholder?: string
//   required: boolean
//   order: number
//   validation?: {
//     minLength?: number, maxLength?: number,
//     min?: number, max?: number,
//     allowedFormats?: string[], maxSizeMB?: number
//     options?: { label: string, value: string }[]
//   }
//   previewConfig?: {
//     canvasX: number, canvasY: number,
//     maxWidth: number, maxHeight: number,
//     fontSize?: number, fontFamily?: string, color?: string
//   }
// }

model DesignTemplate {
  id           String   @id @default(uuid())
  productId    String
  product      Product  @relation(fields: [productId], references: [id])
  name         String
  canvasConfig Json     // background, overlay, dimensions
```

```
  widthPx      Int
  heightPx     Int
}

model UserDesign {
  id              String   @id @default(uuid())
  userId          String
  user            User     @relation(fields: [userId], references: [id])
  productId       String
  name            String
  customizationData Json
  previewUrl      String?
  createdAt       DateTime @default(now())
}

// ─────────────────────────────────────────────
// CART & ORDERS
// ─────────────────────────────────────────────

model Cart {
  id        String     @id @default(uuid())
  userId    String     @unique
  user      User       @relation(fields: [userId], references: [id])
  items     CartItem[]
  updatedAt DateTime   @updatedAt
}

model CartItem {
  id              String         @id @default(uuid())
  cartId          String
  cart            Cart           @relation(fields: [cartId], references: [id])
  variantId       String
  variant         ProductVariant @relation(fields: [variantId], references:
[id])
  quantity        Int
  customizationData Json?
  unitPriceInPaise  Int
  @@unique([cartId, variantId])
}

model Order {
  id              String      @id @default(uuid())
  orderNumber     String      @unique
  userId          String
  user            User        @relation(fields: [userId], references: [id])
  addressId       String
  address         Address     @relation(fields: [addressId], references: [id])
```

```
  couponId          String?
  coupon            Coupon?     @relation(fields: [couponId], references: [id])
  status            OrderStatus @default(PENDING)
  subtotalPaise     Int
  discountPaise     Int         @default(0)
  shippingPaise     Int         @default(0)
  totalPaise        Int
  customerNotes     String?
  createdAt         DateTime    @default(now())
  updatedAt         DateTime    @updatedAt
  items             OrderItem[]
  payment           Payment?
  @@index([userId])
  @@index([status])
  @@index([createdAt])
}

model OrderItem {
  id                String         @id @default(uuid())
  orderId           String
  order             Order          @relation(fields: [orderId], references: [id])
  variantId         String
  variant           ProductVariant @relation(fields: [variantId], references:
[id])
  quantity          Int
  unitPriceInPaise  Int
  customizationData Json?          // Immutable snapshot
  previewUrl        String?
}

// ─────────────────────────────────────────────
// PAYMENTS & COUPONS
// ─────────────────────────────────────────────

model Payment {
  id                String        @id @default(uuid())
  orderId           String        @unique
  order             Order         @relation(fields: [orderId], references: [id])
  razorpayOrderId   String        @unique
  razorpayPaymentId String?
  razorpaySignature String?
  amountInPaise     Int
  currency          String        @default("INR")
  status            PaymentStatus @default(CREATED)
  method            String?
  capturedAt        DateTime?
  createdAt         DateTime      @default(now())
```

```
}

model Coupon {
  id              String     @id @default(uuid())
  code            String     @unique
  type            CouponType
  value           Decimal
  minOrderPaise   Int?
  usageLimit      Int?
  usedCount       Int        @default(0)
  validFrom       DateTime
  validTo         DateTime
  isActive        Boolean    @default(true)
  orders          Order[]
}
```

10.2 Critical Indexes

sql

```sql
-- Full-text search on products
CREATE INDEX idx_products_fts ON products
  USING gin(to_tsvector('english', name || ' ' || description));

-- Order lookups (most frequent admin query)
CREATE INDEX idx_orders_status_created ON orders(status, created_at DESC);

-- Inventory check (prevent oversell)
CREATE UNIQUE INDEX idx_variant_sku ON product_variants(sku);
CREATE INDEX idx_variant_stock ON product_variants(product_id, stock)
  WHERE stock > 0;

-- Cart performance
CREATE UNIQUE INDEX idx_cart_item_unique ON cart_items(cart_id, variant_id);

-- Analytics rollup queries
CREATE INDEX idx_payments_captured ON payments(captured_at)
  WHERE status = 'CAPTURED';

-- Notification fan-out
CREATE INDEX idx_notifications_user_unread ON notifications(user_id, created_at DESC
  WHERE is_read = false;
```

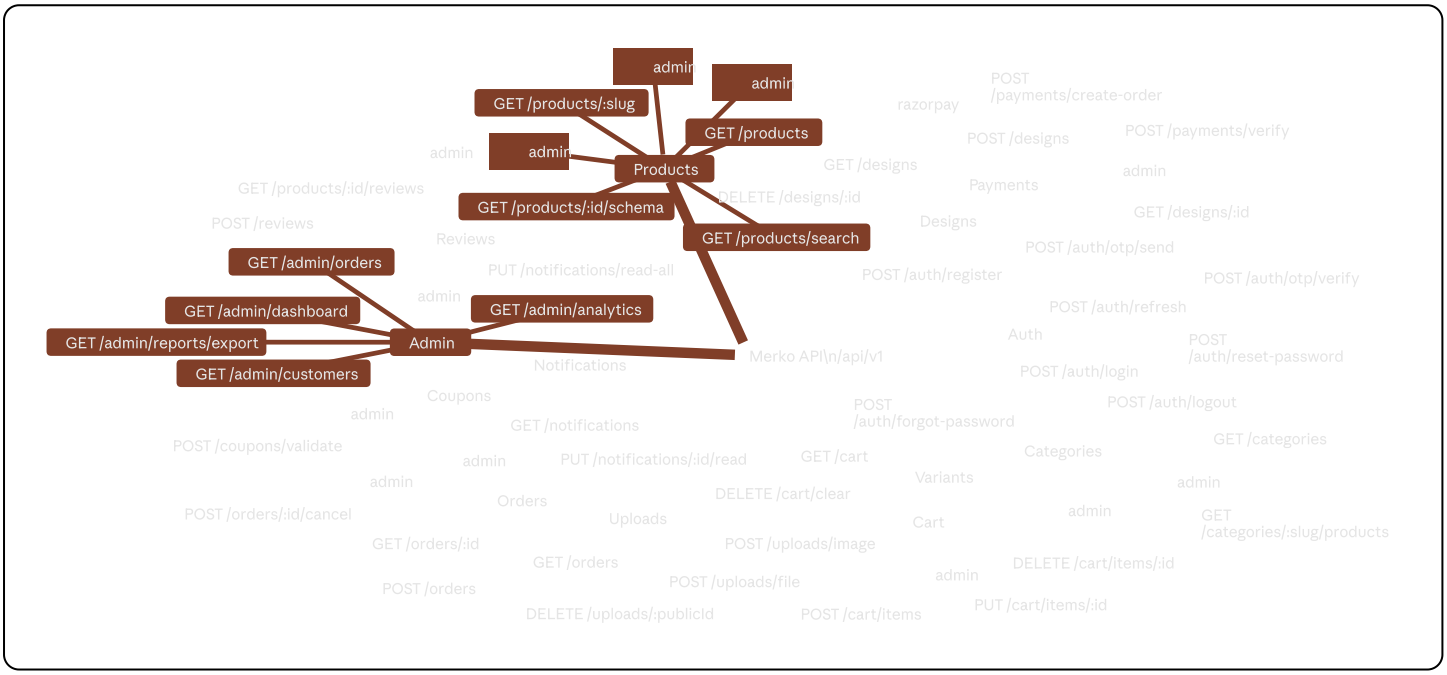
---

11. API Architecture

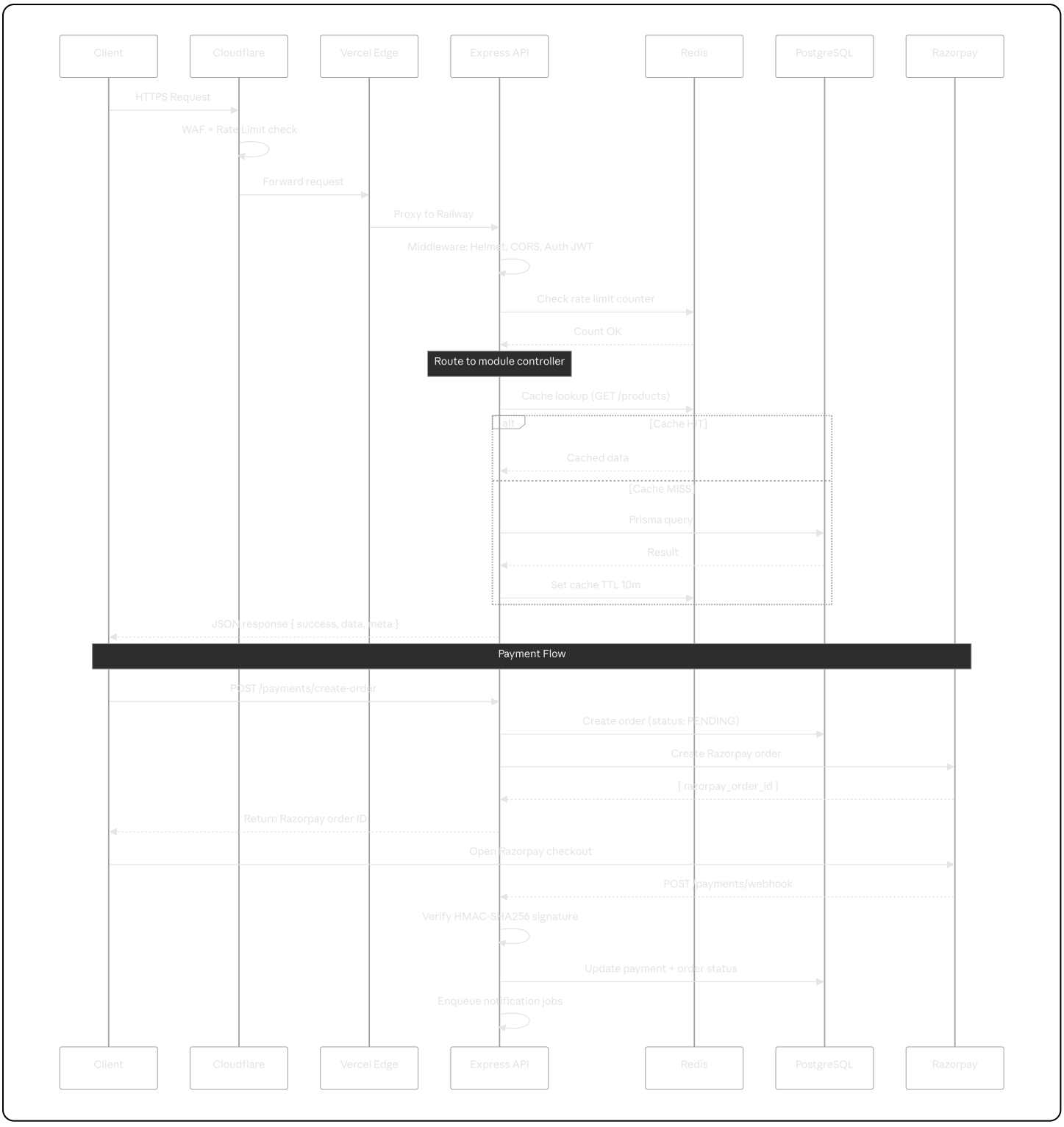
11.1 API Design Principles

- RESTful resource-based URLs
- Versioned: `/api/v1/`
- Standard response envelope: `{ success, data, error, meta }`
- Pagination: cursor-based for feeds, offset for admin grids
- Idempotency keys required on all mutating payment calls
- Rate-limit headers on every response: `X-RateLimit-Limit`, `X-RateLimit-Remaining`

11.2 Complete API Endpoint Map

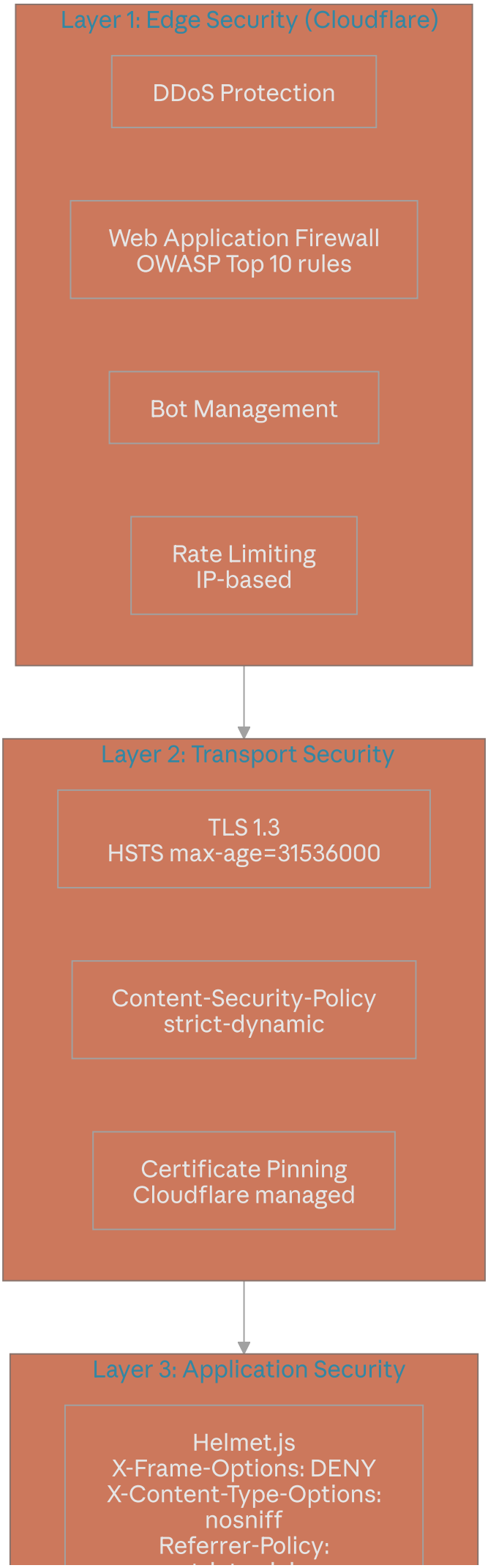


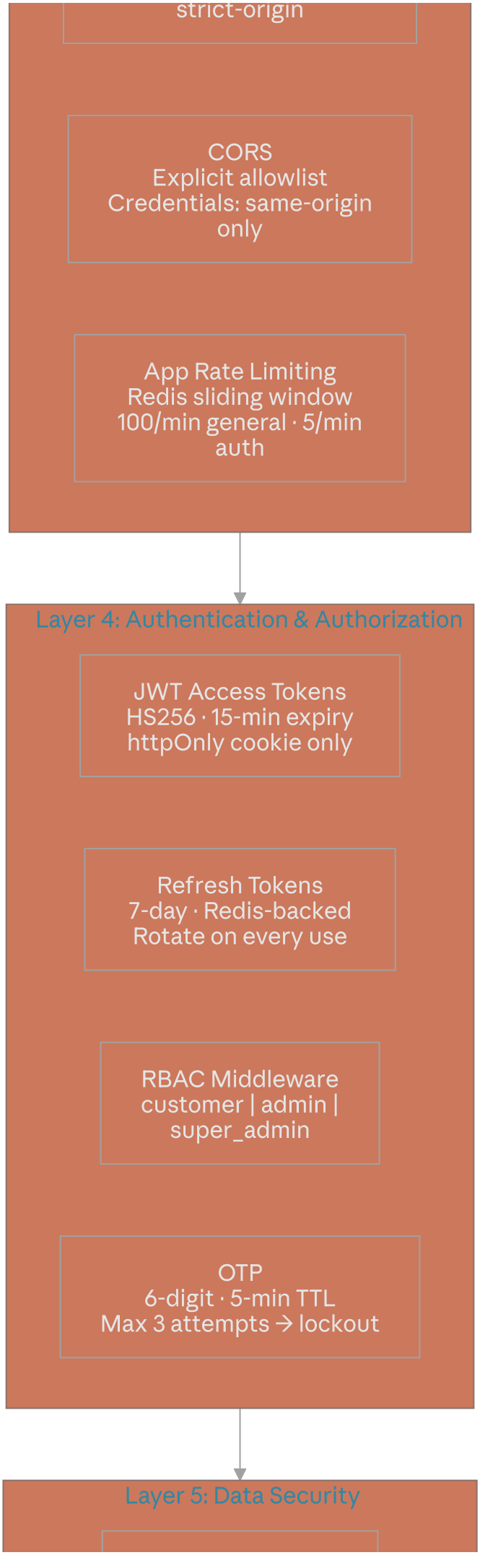
11.3 Request / Response Flow

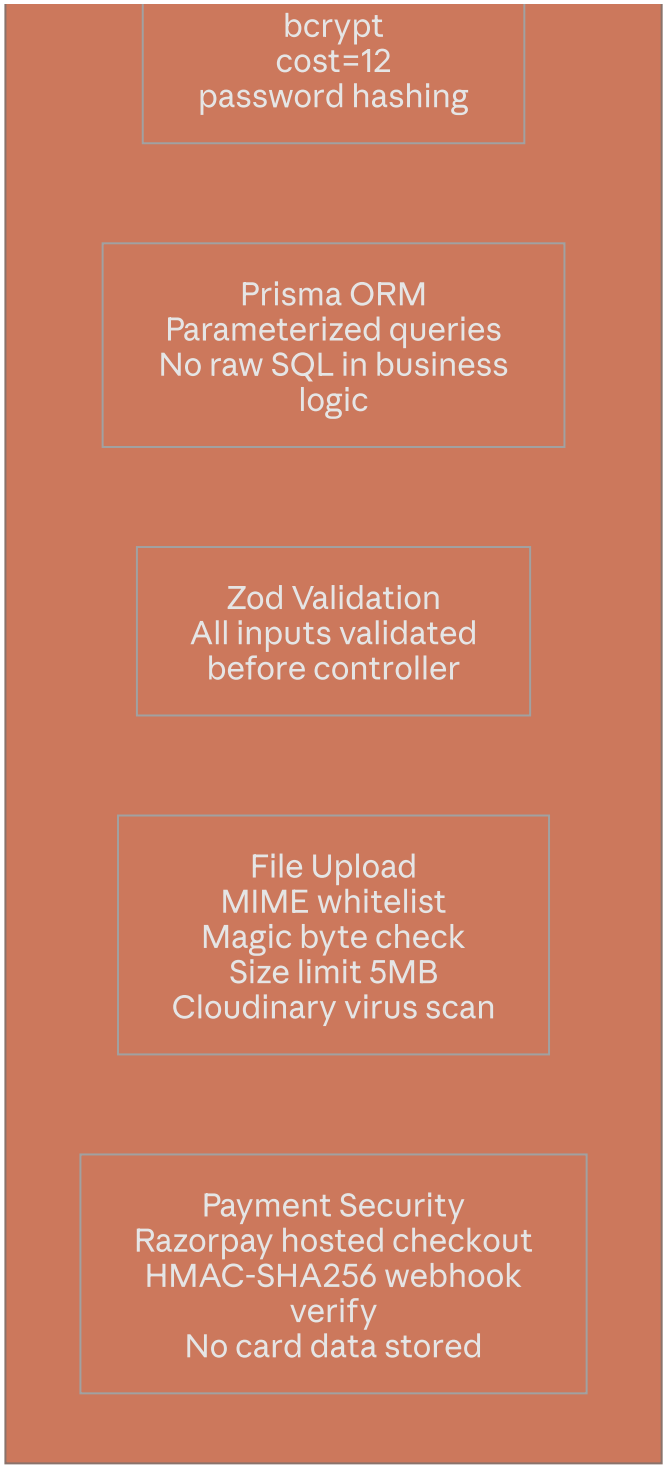


12. Security Architecture

12.1 Security Layers


Layer 1: Edge Security (Cloudflare)
DDoS Protection
Web Application Firewall
OWASP Top 10 rules
Bot Management
Rate Limiting
IP-based
Layer 2: Transport Security
TLS 1.3
HSTS max-age=31536000
Content-Security-Policy
strict-dynamic
Certificate Pinning
Cloudflare managed
Layer 3: Application Security
Helmet.js
X-Frame-Options: DENY
X-Content-Type-Options:
nosniff
Referrer-Policy:


strict-origin
CORS
Explicit allowlist
Credentials: same-origin only
App Rate Limiting
Redis sliding window
100/min general · 5/min auth
Layer 4: Authentication & Authorization
JWT Access Tokens
HS256 · 15-min expiry
httpOnly cookie only
Refresh Tokens
7-day · Redis-backed
Rotate on every use
RBAC Middleware
customer | admin | super_admin
OTP
6-digit · 5-min TTL
Max 3 attempts → lockout
Layer 5: Data Security

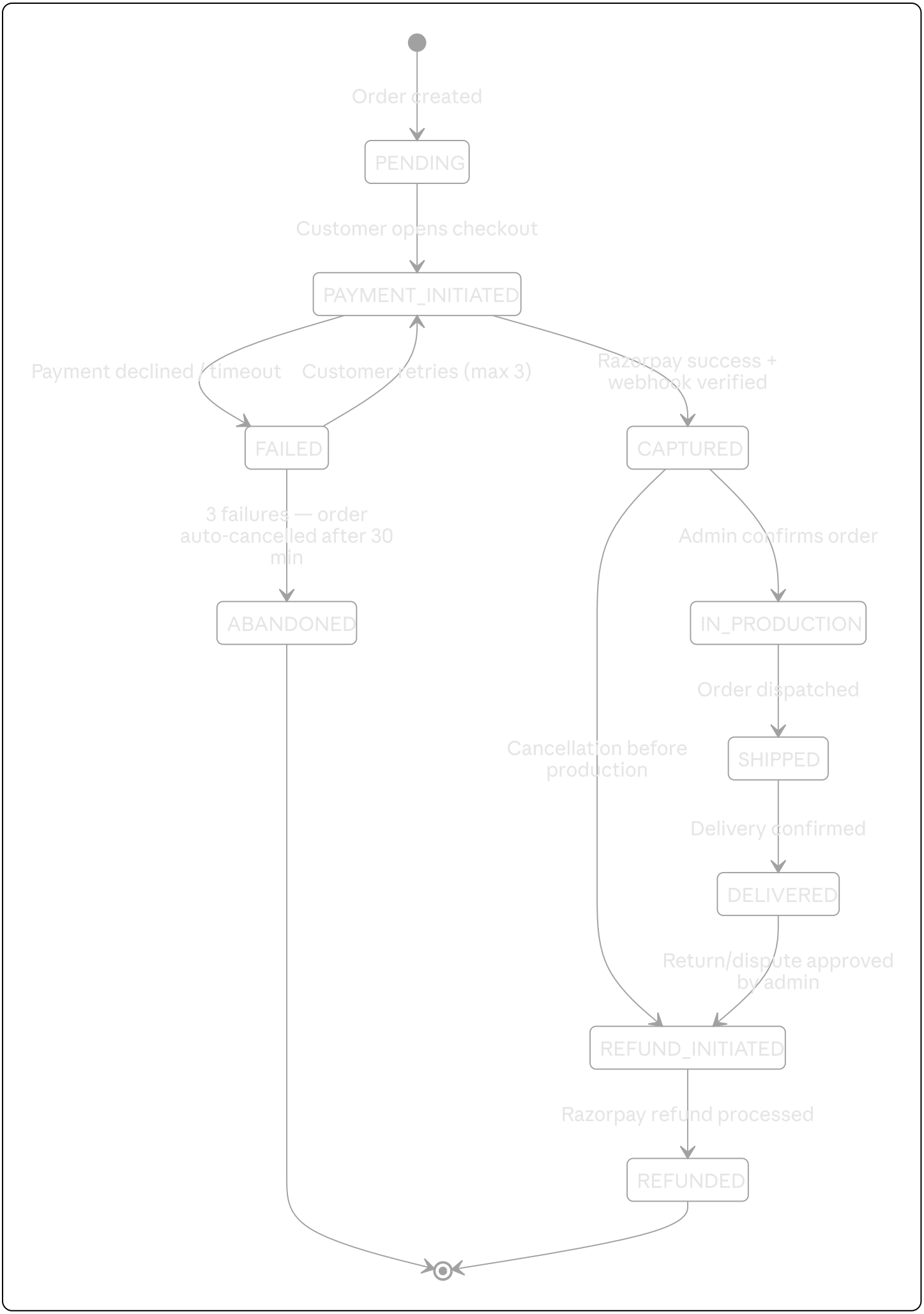

bcrypt
cost=12
password hashing
Prisma ORM
Parameterized queries
No raw SQL in business logic
Zod Validation
All inputs validated before controller
File Upload
MIME whitelist
Magic byte check
Size limit 5MB
Cloudinary virus scan
Payment Security
Razorpay hosted checkout
HMAC-SHA256 webhook verify
No card data stored

13.2 Cloudinary Folder Structure & Policy

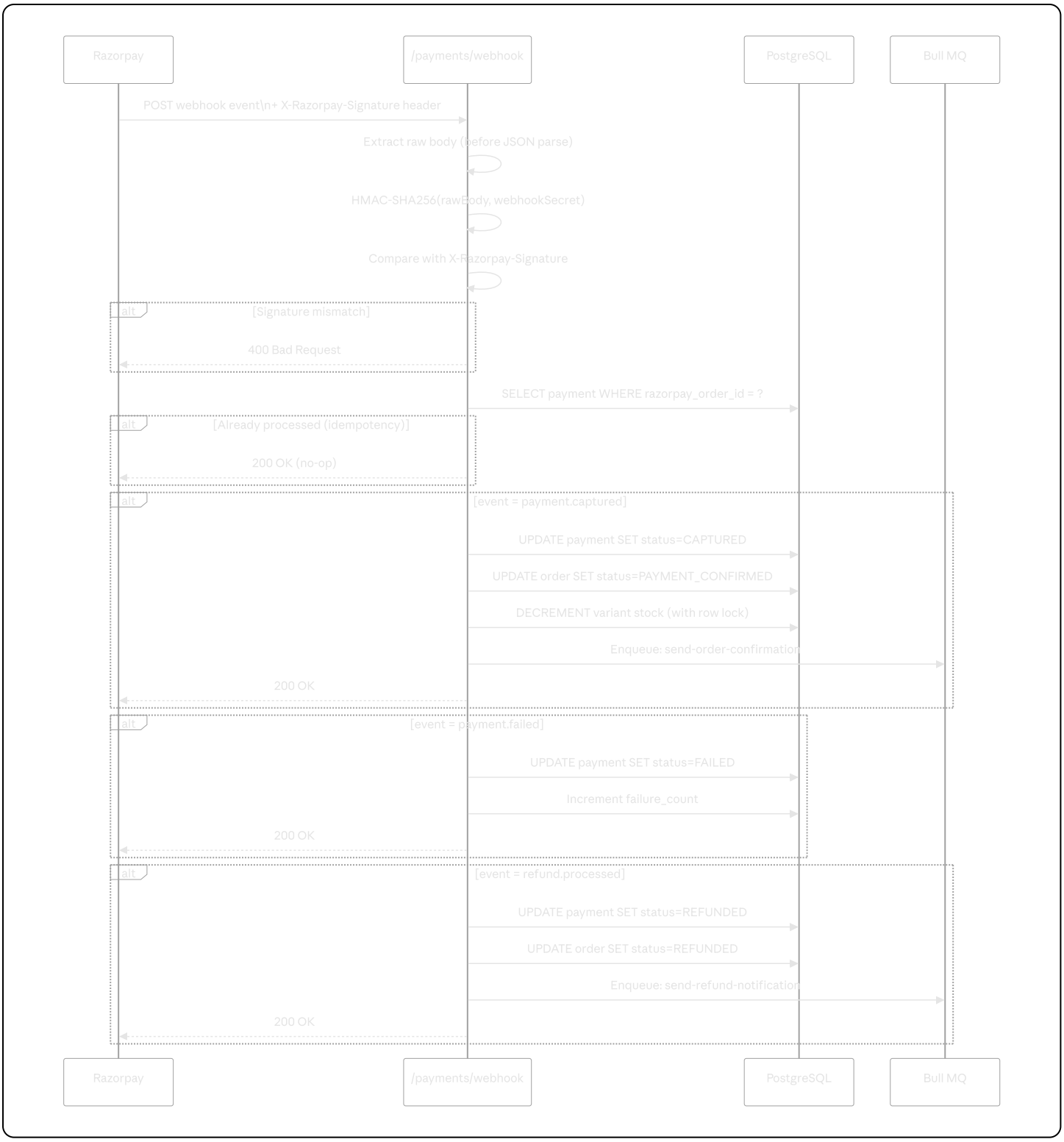
Folder	Content	Access	Retention
`/merko/products/`	Product catalog images	Public	Indefinite
`/merko/user-uploads/`	Customer customization files	Signed URL only	1 year
`/merko/previews/`	Generated design previews	Public (UUID in URL)	2 years
`/merko/gallery/`	Admin portfolio uploads	Public	Indefinite
`/merko/temp/`	Processing intermediates	Private	24 hours (auto-purge)

14. Payment Architecture

14.1 Payment Lifecycle

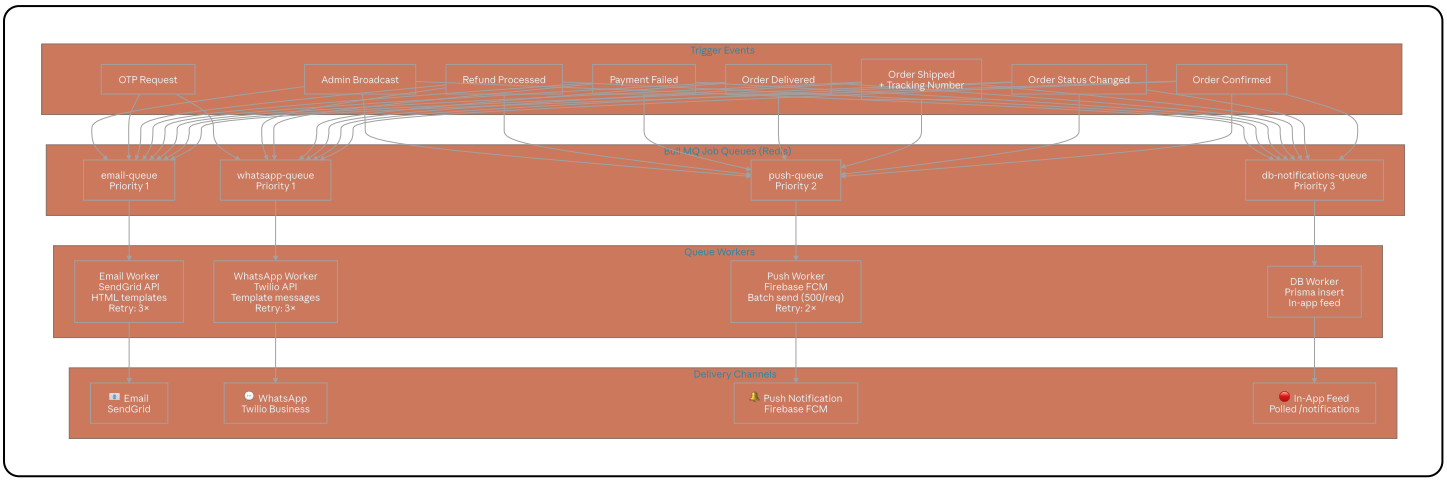


14.2 Webhook Security Flow


Razorpay
/payments/webhook
PostgreSQL
Bull MQ
POST webhook event\n+ X-Razorpay-Signature header
Extract raw body (before JSON parse)
HMAC-SHA256(rawBody, webhookSecret)
Compare with X-Razorpay-Signature
alt
[Signature mismatch]
400 Bad Request
SELECT payment WHERE razorpay_order_id = ?
alt
[Already processed (idempotency)]
200 OK (no-op)
alt
[event = payment.captured]
UPDATE payment SET status=CAPTURED
UPDATE order SET status=PAYMENT_CONFIRMED
DECREMENT variant stock (with row lock)
Enqueue: send-order-confirmation
200 OK
alt
[event = payment.failed]
UPDATE payment SET status=FAILED
Increment failure_count
200 OK
alt
[event = refund.processed]
UPDATE payment SET status=REFUNDED
UPDATE order SET status=REFUNDED
Enqueue: send-refund-notification
200 OK
Razorpay
/payments/webhook
PostgreSQL
Bull MQ

15. Notification Architecture

15.1 Notification Topology

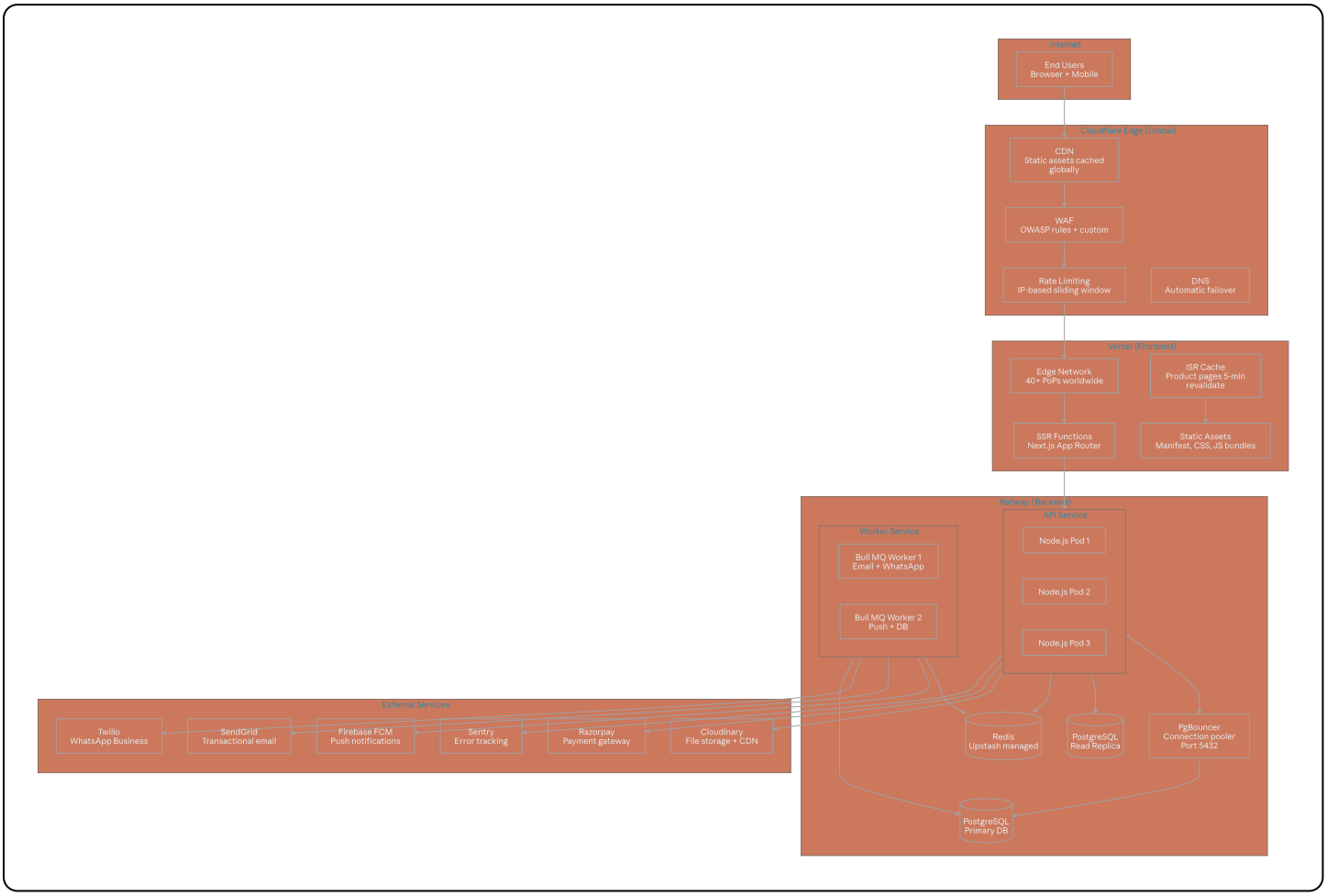


15.2 Notification Templates

Event	Email	WhatsApp	Push
Registration OTP	✓	✓	✗
Order Confirmed	✓	✓	✓
In Production	✓	✓	✓
Shipped	✓	✓	✓
Delivered	✓	✓	✓
Payment Failed	✓	✓	✓
Refund Processed	✓	✓	✓
Coupon Alert	✓	✗	✓
Review Request (+3 days)	✓	✗	✓

16. Deployment Architecture

16.1 Infrastructure Overview



16.2 Environment Configuration

Environment	Frontend	Backend	Database	Purpose
`development`	`localhost:3000`	`localhost:4000`	Local Docker PostgreSQL	Local dev
`preview`	Vercel PR preview	Railway PR env	Shared staging DB	PR review
`staging`	`staging.merko.in`	Railway staging	Staging DB (copy of prod schema)	QA + regression
`production`	`merko.in`	Railway production	Production DB	Live

16.3 Docker Compose (Development)

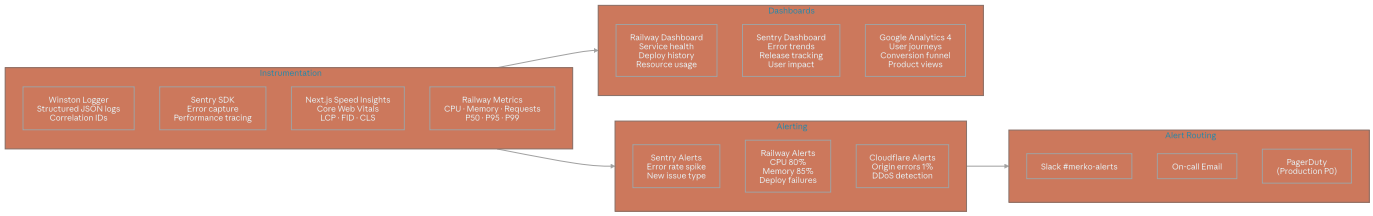
yaml

```
# High-level service definitions (no code, description only)
services:
  postgres:    Image: postgres:16, Port: 5432, Volume: pgdata
  redis:       Image: redis:7-alpine, Port: 6379
  api:         Build: ./apps/api, Port: 4000, Depends: postgres + redis
  worker:      Build: ./apps/api, CMD: worker, Depends: postgres + redis
```

---

17. Monitoring Architecture

17.1 Observability Stack



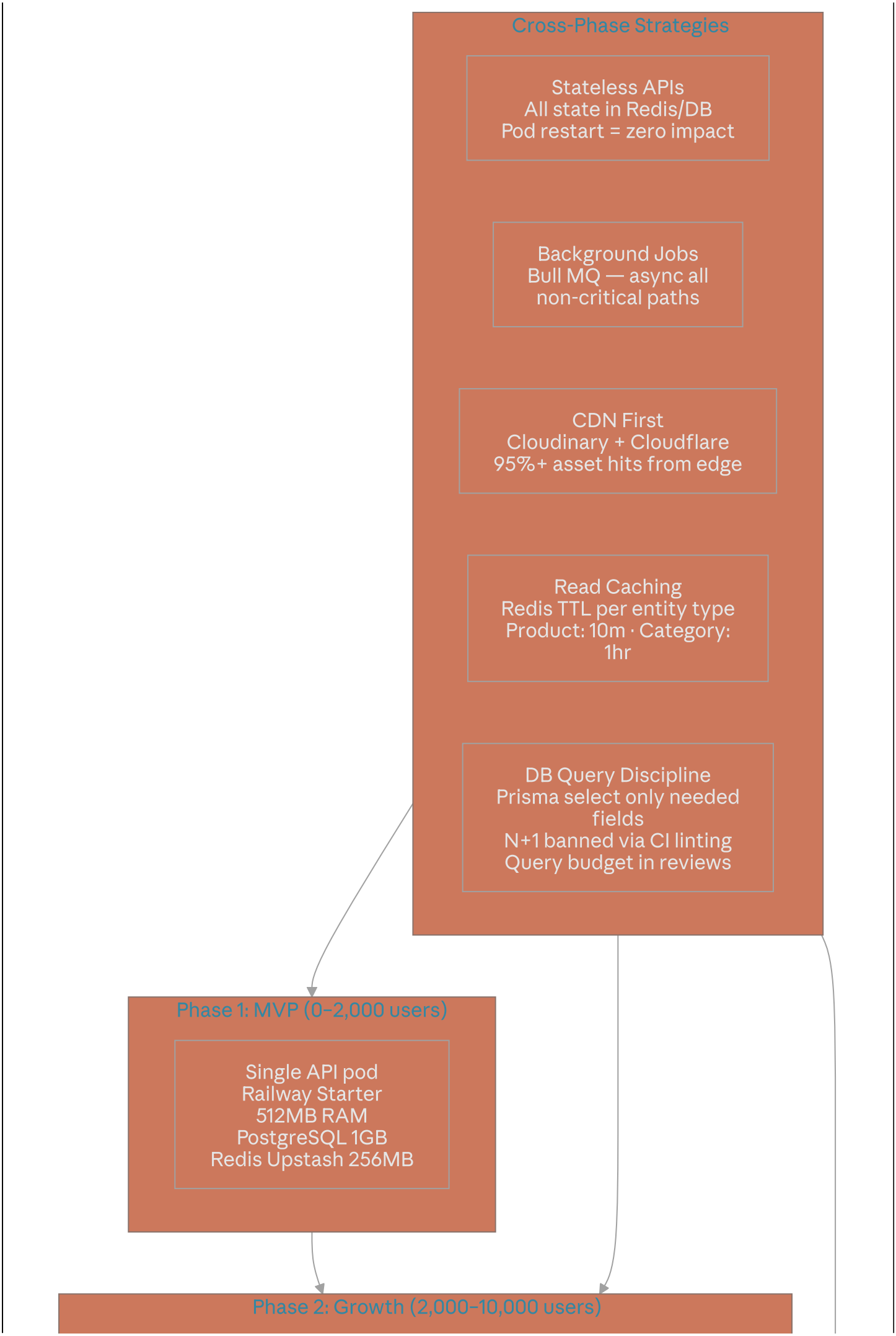
17.2 Key Alert Thresholds

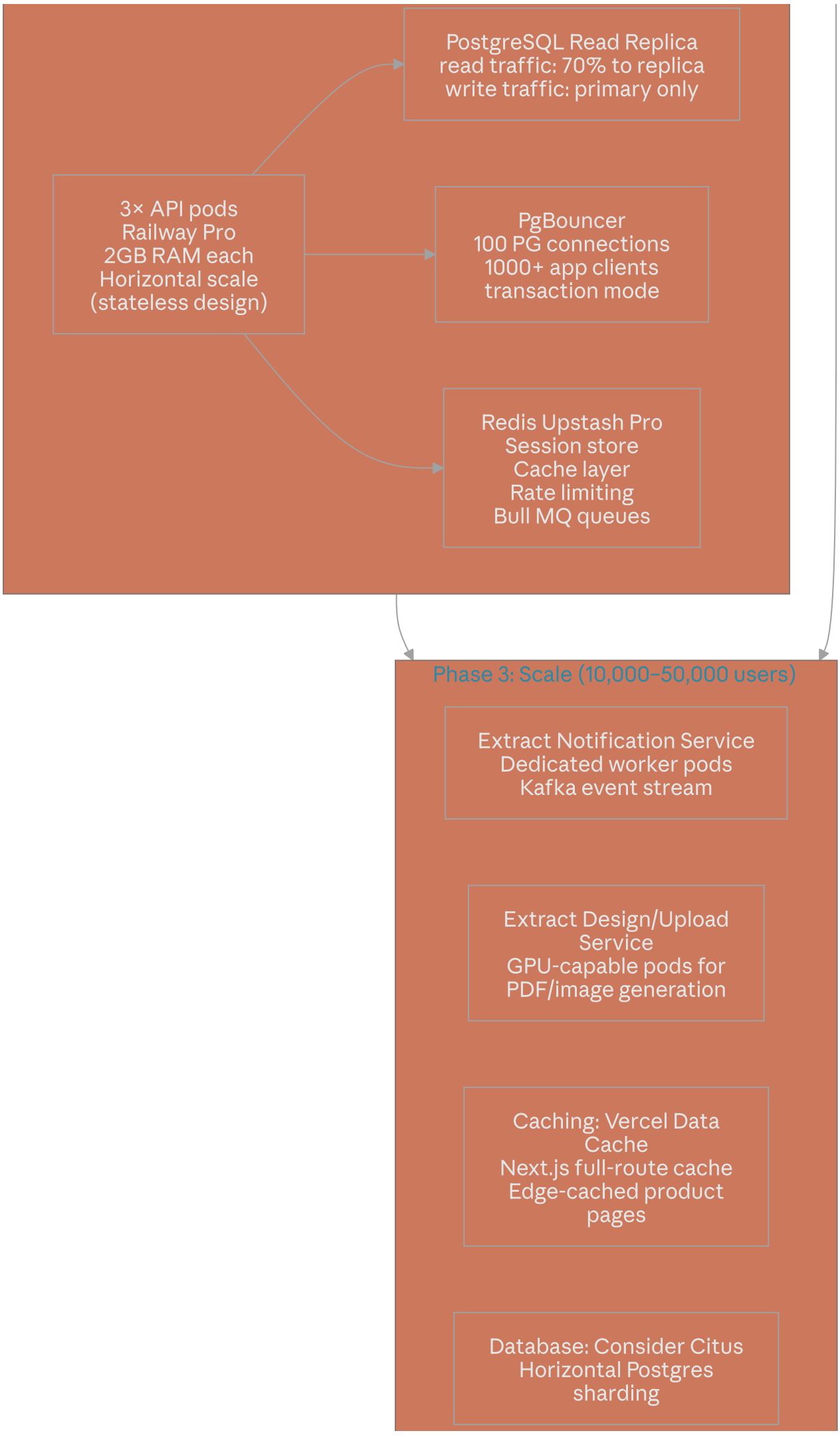
Alert	Threshold	Severity	Channel
API error rate	> 1% over 5 min	P1	Slack + Email
API P99 latency	> 2000ms	P1	Slack
DB connection pool saturation	> 90%	P0	Slack + PagerDuty
Payment webhook failures	Any	P0	Slack + PagerDuty
Failed deploys	Any	P1	Slack
Disk usage	> 80%	P2	Email
Uptime check failure	2 consecutive fails	P0	All channels

---

18. Scalability Strategy

18.1 Scaling Architecture for 10,000+ Users


Cross-Phase Strategies
Stateless APIs
All state in Redis/DB
Pod restart = zero impact
Background Jobs
Bull MQ — async all non-critical paths
CDN First
Cloudinary + Cloudflare
95%+ asset hits from edge
Read Caching
Redis TTL per entity type
Product: 10m · Category: 1hr
DB Query Discipline
Prisma select only needed fields
N+1 banned via CI linting
Query budget in reviews
Phase 1: MVP (0–2,000 users)
Single API pod
Railway Starter
512MB RAM
PostgreSQL 1GB
Redis Upstash 256MB
Phase 2: Growth (2,000–10,000 users)


PostgreSQL Read Replica
read traffic: 70% to replica
write traffic: primary only
3× API pods
Railway Pro
2GB RAM each
Horizontal scale
(stateless design)
PgBouncer
100 PG connections
1000+ app clients
transaction mode
Redis Upstash Pro
Session store
Cache layer
Rate limiting
Bull MQ queues
Phase 3: Scale (10,000–50,000 users)
Extract Notification Service
Dedicated worker pods
Kafka event stream
Extract Design/Upload Service
GPU-capable pods for PDF/image generation
Caching: Vercel Data Cache
Next.js full-route cache
Edge-cached product pages
Database: Consider Citus
Horizontal Postgres sharding

19.2 Branch Strategy

```
gitGraph
   commit id: "Initial commit"
   branch develop
   checkout develop
   commit id: "feat: auth module"
   branch feature/product-catalog
   checkout feature/product-catalog
   commit id: "feat: product CRUD"
   commit id: "feat: search + filters"
   checkout develop
   merge feature/product-catalog id: "merge: product catalog"
   branch feature/customization-engine
   checkout feature/customization-engine
   commit id: "feat: dynamic field builder"
   commit id: "feat: live preview canvas"
   checkout develop
   merge feature/customization-engine id: "merge: customization engine"
   branch release/v1.0.0
   checkout release/v1.0.0
   commit id: "chore: bump version 1.0.0"
   commit id: "fix: payment webhook edge case"
   checkout main
   merge release/v1.0.0 id: "release: v1.0.0" tag: "v1.0.0"
   checkout develop
   merge main id: "sync: post-release"
```

19.3 Deployment Checklist

Pre-deployment

- [ ] All CI quality gates pass
- [ ] Migration script reviewed and tested on staging
- [ ] Environment variables verified in Railway secrets
- [ ] Razorpay webhook endpoint registered for new domain/path changes
- [ ] Cloudinary allowed origins updated if needed
- [ ] Load test run against staging (k6 — 500 VUs, 5 min ramp)

Post-deployment

- [ ] `/api/v1/health` returns 200 on all pods
- [ ] Sentry release tracking shows new version deployed
- [ ] Place test order end-to-end (checkout → payment → confirmation)
- [ ] Check Bull MQ dashboard — queues draining normally

- [ ] Verify Railway metrics show normal CPU/memory baseline

Appendix A: Technology Decision Log

Decision	Chosen	Alternatives Considered	Rationale
ORM	Prisma	TypeORM, Drizzle, Knex	Best DX, type safety, migration tooling, schema-first
Frontend	Next.js 15	Remix, Nuxt	SSR/ISR, Vercel integration, ecosystem
Component library	Shadcn UI	MUI, Chakra, Ant Design	Composable, no lock-in, Tailwind-native
Background jobs	Bull MQ	Agenda, BullMQ, SQS	Redis-backed, reliable, excellent DX
State management	Zustand + TanStack Query	Redux, Jotai	Minimal boilerplate, server/client state separation
Payment	Razorpay	Stripe, PayU, CCAvenue	Best India UPI support, competitive fees
Storage	Cloudinary	AWS S3, Uploadcare	Built-in transformations, CDN, generous free tier
Hosting (API)	Railway	Render, Fly.io, EC2	Easy PostgreSQL managed, zero DevOps, good pricing
Error monitoring	Sentry	Datadog, New Relic, Rollbar	Best DX, generous free tier, excellent Next.js integration

Appendix B: Customization Field Schema Reference

```json
```

```
{
  "fields": [
    {
      "id": "uuid-v4",
      "type": "text",
      "label": "Employee Name",
      "placeholder": "Enter full name",
      "required": true,
      "order": 1,
      "validation": {
        "minLength": 2,
        "maxLength": 50
      },
      "previewConfig": {
        "canvasX": 120,
        "canvasY": 200,
        "maxWidth": 200,
        "fontSize": 16,
        "fontFamily": "Inter",
        "color": "#1a1a1a"
      }
    },
    {
      "id": "uuid-v4",
      "type": "image",
      "label": "Employee Photo",
      "required": true,
      "order": 2,
      "validation": {
        "allowedFormats": ["image/jpeg", "image/png"],
        "maxSizeMB": 2
      },
      "previewConfig": {
        "canvasX": 20,
        "canvasY": 60,
        "maxWidth": 80,
        "maxHeight": 96
      }
    },
    {
      "id": "uuid-v4",
      "type": "dropdown",
      "label": "Department",
      "required": false,
      "order": 3,
      "validation": {
```

```
        "options": [
          { "label": "Engineering", "value": "engineering" },
          { "label": "Sales", "value": "sales" },
          { "label": "HR", "value": "hr" }
        ]
      }
    }
  ]
}
```

Document generated for Merko v1.0 — Last updated: June 2026

Review cycle: Before each major release